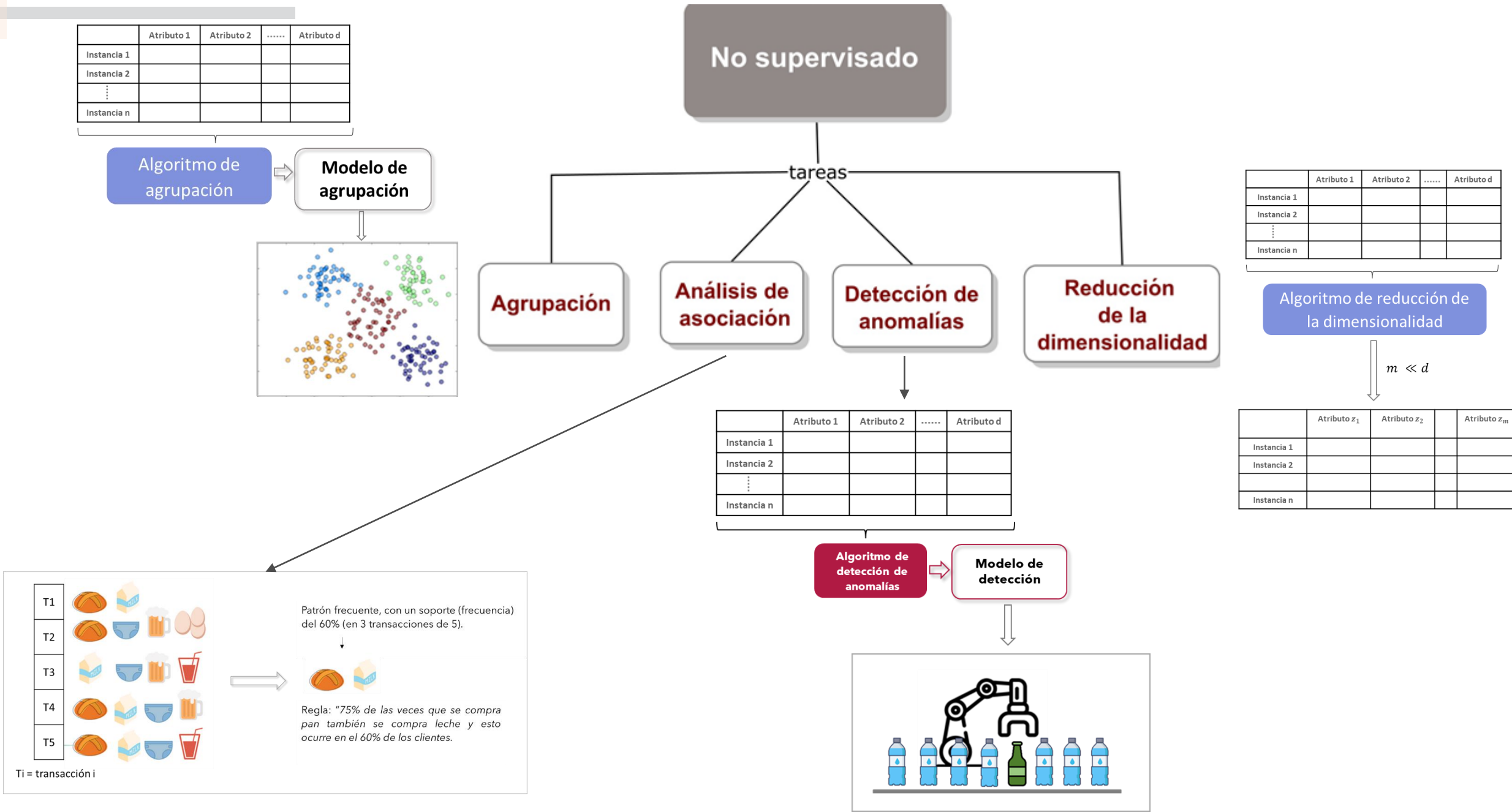


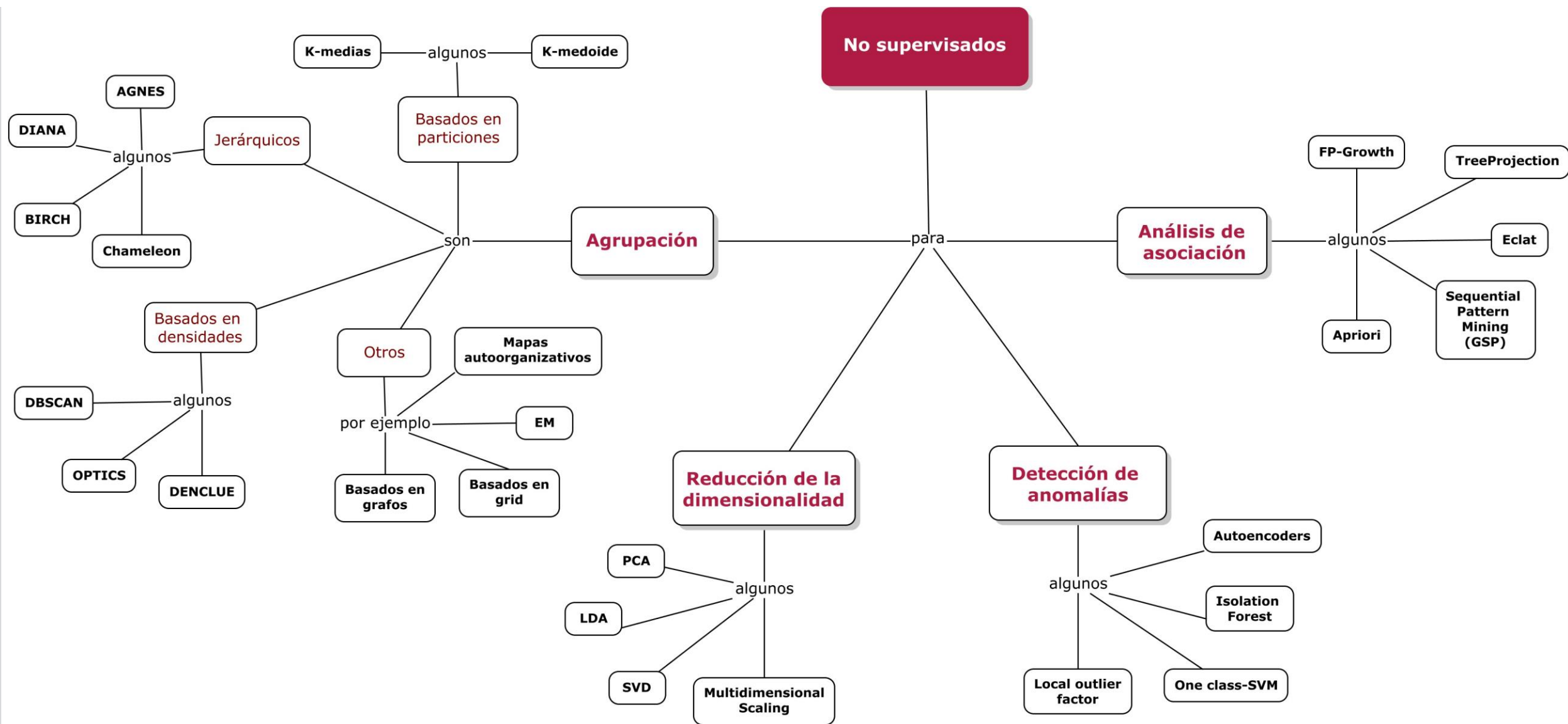
Sesión 7. Aprendizaje no supervisado. Agrupación.

- Aprendizaje no supervisado. Algunos ejemplos
- La tarea de agrupación
- Algoritmo K-medias
- Evaluación de modelos de agrupación
- Interpretación de modelos de agrupación

Tareas no supervisadas



Algunos algoritmos no supervisados:



La tarea de agrupación

- **Grupo:** colección de objetos que comparten características comunes.

Atributo ₁	Atributo ₂	Atributo ₃	Atributo ₄	Atributo ₅	Atributo ₅		Atributo _d
⋮							

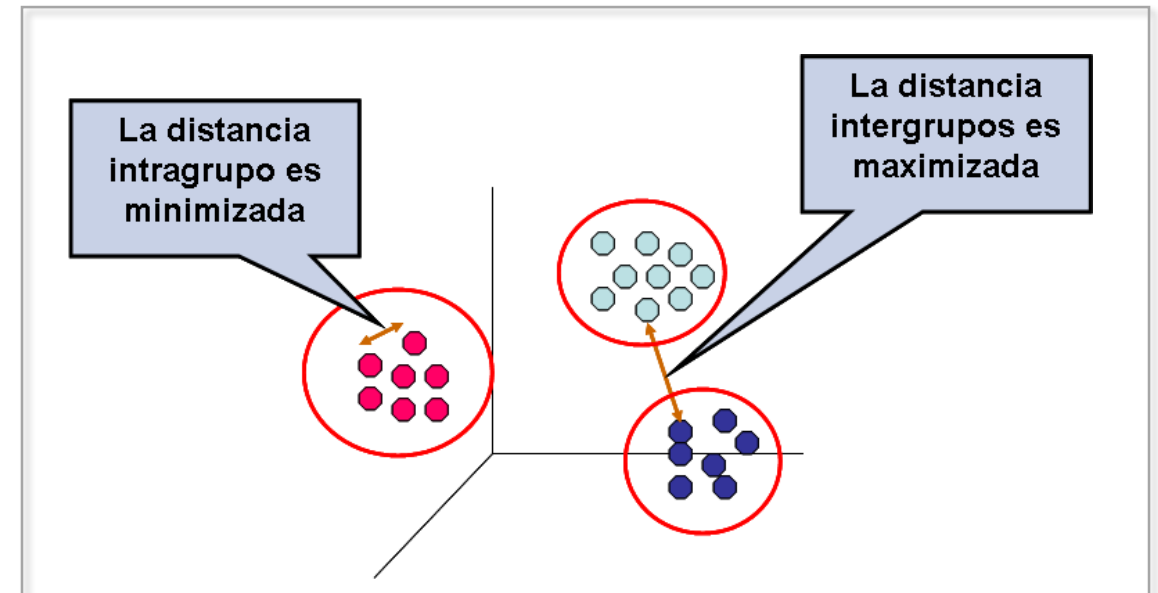
Un grupo puede ser considerado como una clase implícita. Por lo tanto, la agrupación puede ser visualizada como una forma de clasificación, pero no supervisada.

También se conoce como segmentación de datos.

Los datos son agrupados utilizando sólo la información que describe a los datos y sus relaciones.

Objetos dentro de un grupo deben ser similares (o estar relacionados), y diferentes (o no relacionados) a objetos de otros grupos.

Se utilizan medidas o criterios de similitud, que dependerán del tipo de variable presente en el conjunto de datos.

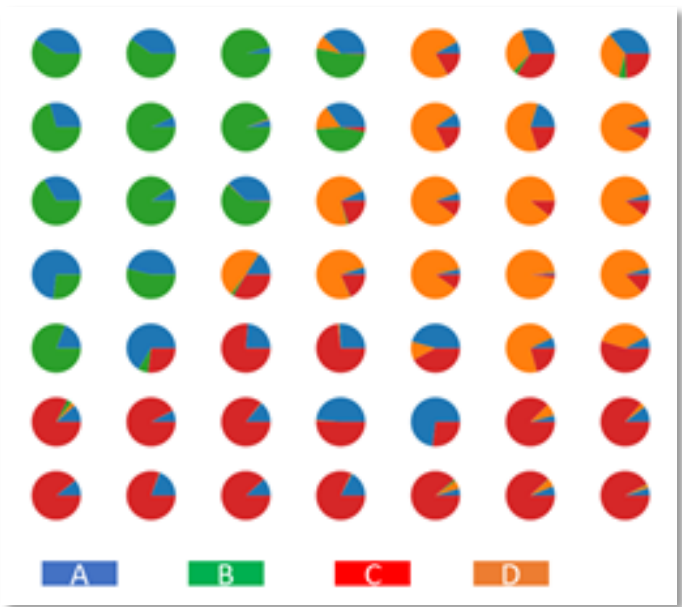


Ejemplos de aplicación

1. Aprendizaje no supervisado en el perfilamiento de clientes para *profit scoring* - Caso de estudio de una Fintech Latinoamericana.

María Juliana Moya

- **Objetivo.** Encontrar patrones intrínsecos entre las variables sociodemográficas, comportamentales y financieras que permitan realizar la identificación de los clientes a los cuales otorgar un microcrédito, para luego realizar un perfilamiento detallado que apoye la toma de decisiones en la organización.
- **Datos.** Los datos se obtuvieron a partir de las bases de datos de la compañía. Con el fin de evitar el manejo de información confidencial, todos los datos fueron anonimizados.
- **Algoritmos.** Mapas autoorganizativos de Kohonen (SOM) y K-medias (esquema híbrido)



Modelo SOM coloreado con K-medias



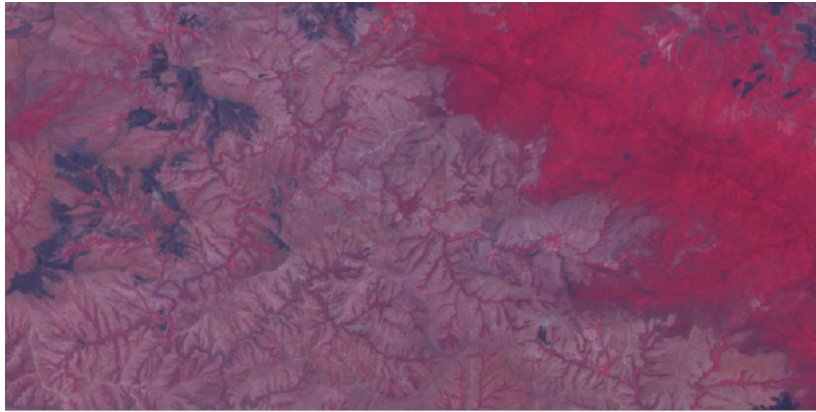
		Naranja	Rojo	Verde	Azúl
Estado del último crédito	Mora	31,60%	32,70%	42,70%	35,80%
	Cancelado	46,80%	41,40%	33,70%	36,20%
	Vigente	21,6%	26,0%	23,1%	28%
Monto promedio solicitado	Min	\$ 200.000	\$ 200.000	\$ 200.000	\$ 200.000
	Max	\$ 2.320.500	\$ 1.980.000	\$ 1.855.000	\$ 2.859.000
	Media	\$ 427.520	\$ 362.330	\$ 372.360	\$ 377.920
Cantidad de créditos solicitados	Min	0,0	0,0	0,0	0,0
	Max	20,0	10,0	8,0	12,0
	Media	2,1	2,4	1,7	2,3
Días de mora promedio	Min	0	0	0	0
	Max	180	271	320	271
	Media	28,4	37,0	42,0	39,4
Rentabilidad		Alta	Media	No rentable	Baja

Grupo	Características
Alta	Ingresos calculados > \$1,600,000
	Score promedio 632
	Experiencia crediticia > 5 años
	Más de 4 productos financieros vigentes (72%)
Media	Ingresos calculados > \$1,400,000
	Score promedio 672
	Experiencia crediticia > 7 años
	Más de 3 productos financieros vigentes (67%)
Baja	Ingresos calculados entre \$1,100,000 - \$1,500,000
	Score promedio 575
	Experiencia crediticia > 7 años
	Más de 4 productos financieros vigentes (59%)
No rentable	Ingresos calculados entre \$1,000,000 - \$1,300,000
	Score promedio 550
	Experiencia crediticia > 6 años
	Más de 2 productos financieros vigentes (67%)

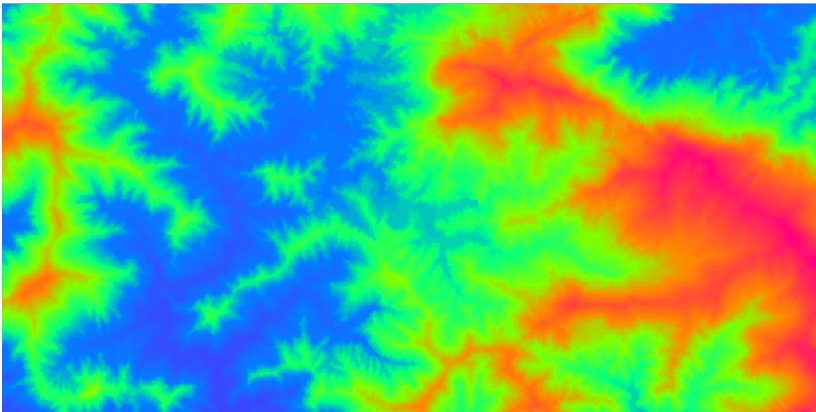
2. Identificación de paisaje a partir de imágenes de satélite y modelos digitales de elevación.

- Objetivo. Inferir propiedades del suelo a partir de la variación del relieve representada por medio de un modelo digital de elevación (MDE) y de las condiciones de la superficie del terreno captadas por imágenes satelitales.
- Datos. Imagen satelital y modelo digital de elevación de la zona de estudio.
- Algoritmos. Agrupación difusa (*fuzzy c-means*)

Imagen satelital

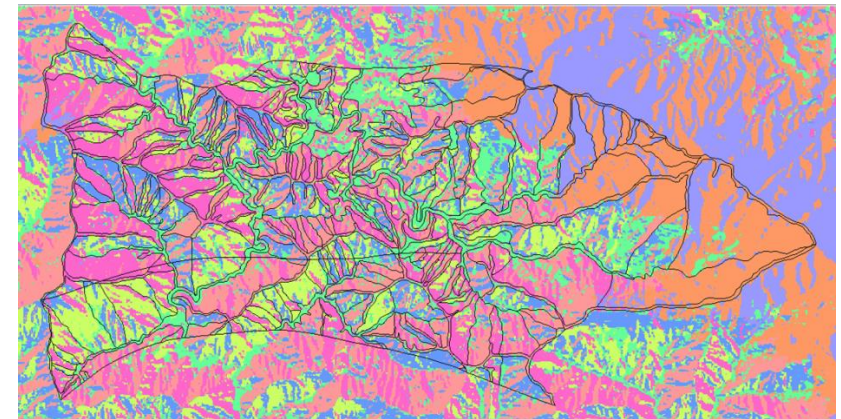
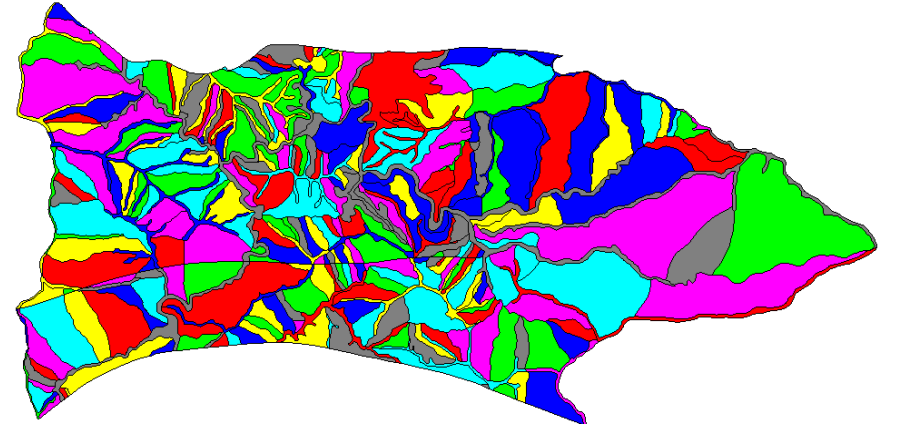


Modelo digital de elevación



Banda XS1
Banda XS2
Banda XS3
NDVI
Aspect
Catchment
Ind_hum
Plang
profile
Slope

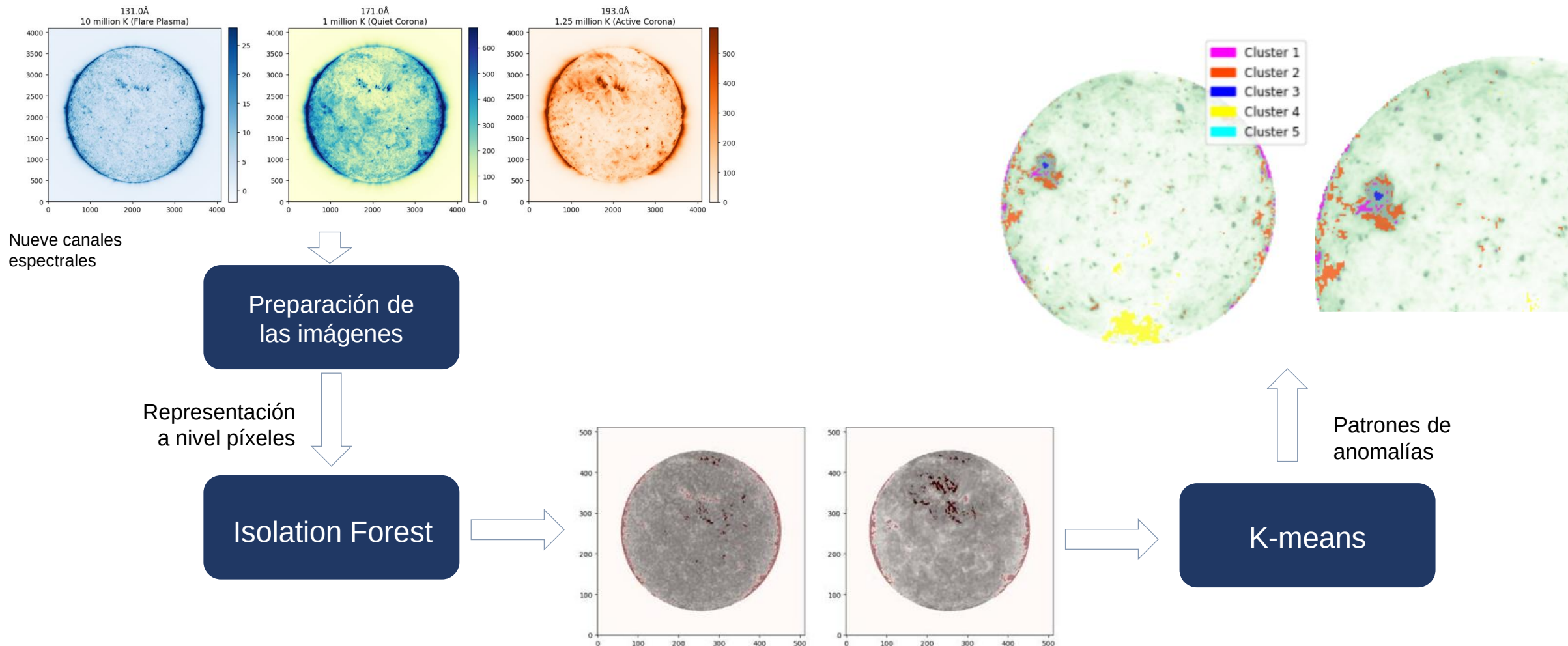
Clasificación realizada por el grupo de expertos



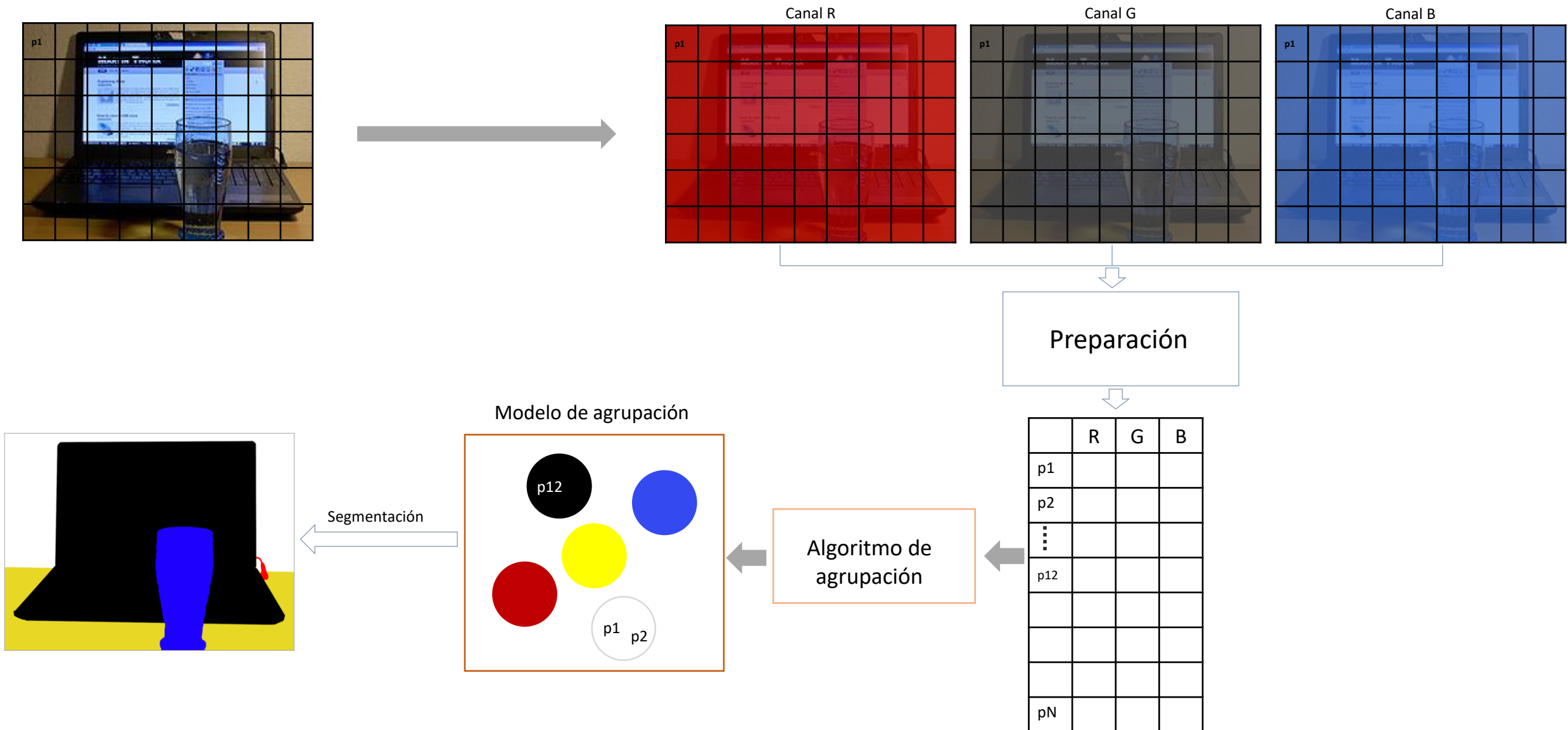
Clasificación realizada por el modelo

3. Solar Observer Learning Anomaly Recognition - Anomaly Detection in SDO/AIA Images

Este trabajo presenta SOLAR (Solar Observer Learning Anomaly Recognition), un pipeline de aprendizaje automático no supervisado para detectar anomalías en imágenes de AIA/SDO aprovechando su capacidad de observación multicanal. El sistema utiliza modelos como Isolation Forest para identificar píxeles inusuales sin necesidad de etiquetas, y algoritmos como K-Means, MiniBatch K-Means y Gaussian Mixture para agruparlos y revelar patrones espaciales coherentes.



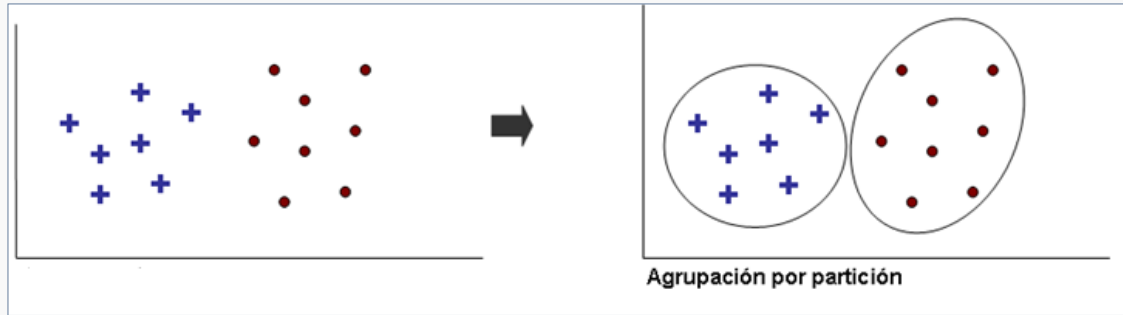
4. Segmentación de imágenes



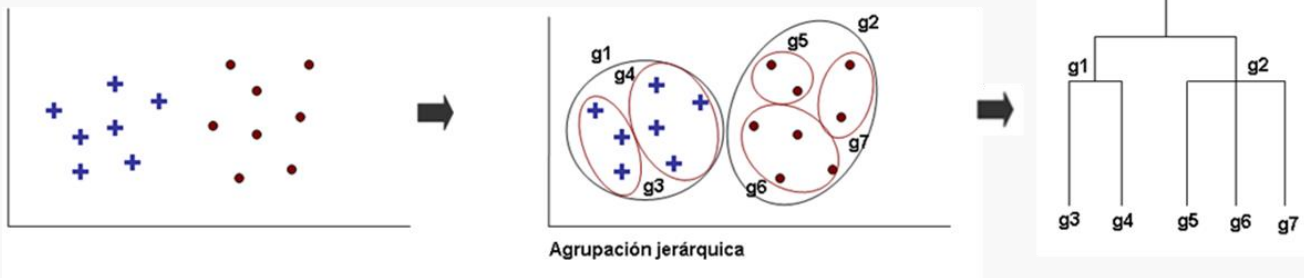
❏ Aspectos de diseño

a) Criterio de división:

Por particiones: Se divide el conjunto de datos en grupos al mismo nivel.



Jerárquico: Conjuntos de grupos anidados que pueden organizarse como un árbol jerárquico (los grupos tienen subgrupos).



b) Separación entre grupos:

- Exclusivo: Cada objeto se asigna a un solo grupo.
- Solapado: Un objeto puede pertenecer simultáneamente a más de un grupo.
- Difuso: Cada objeto pertenece a un grupo o a varios grupos, pero con un grado de pertenencia que está entre 0 y 1. Los grupos pueden ser tratados como conjuntos difusos.

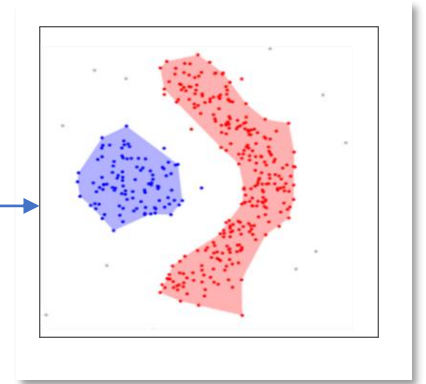
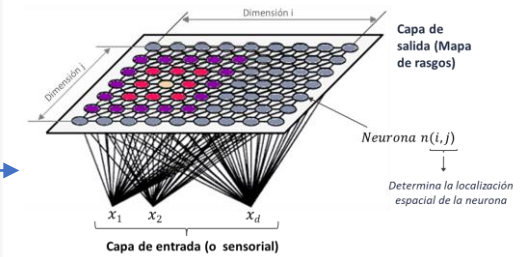
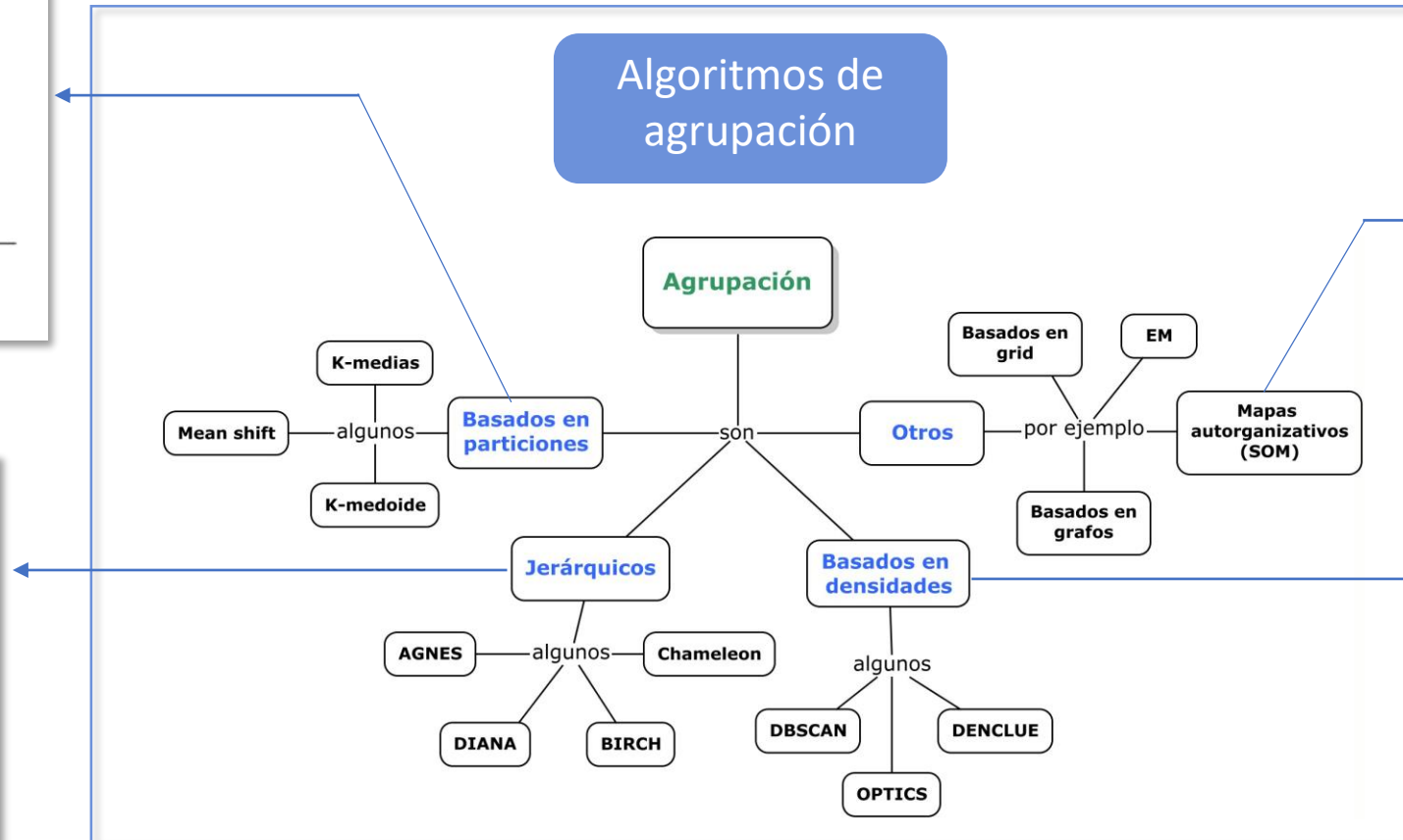
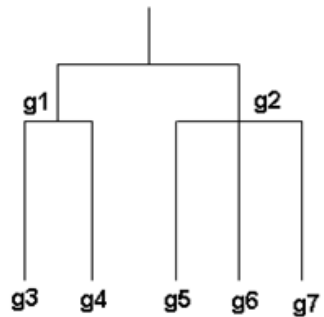
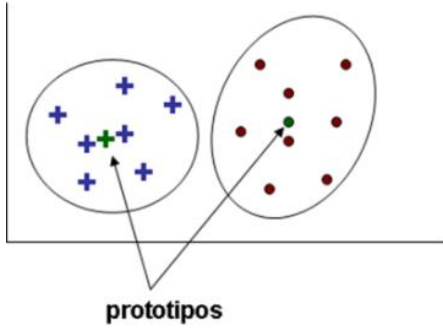
c) Medidas de similitud:

- Por distancia entre objetos
- Por densidad o contigüidad (conectividad)

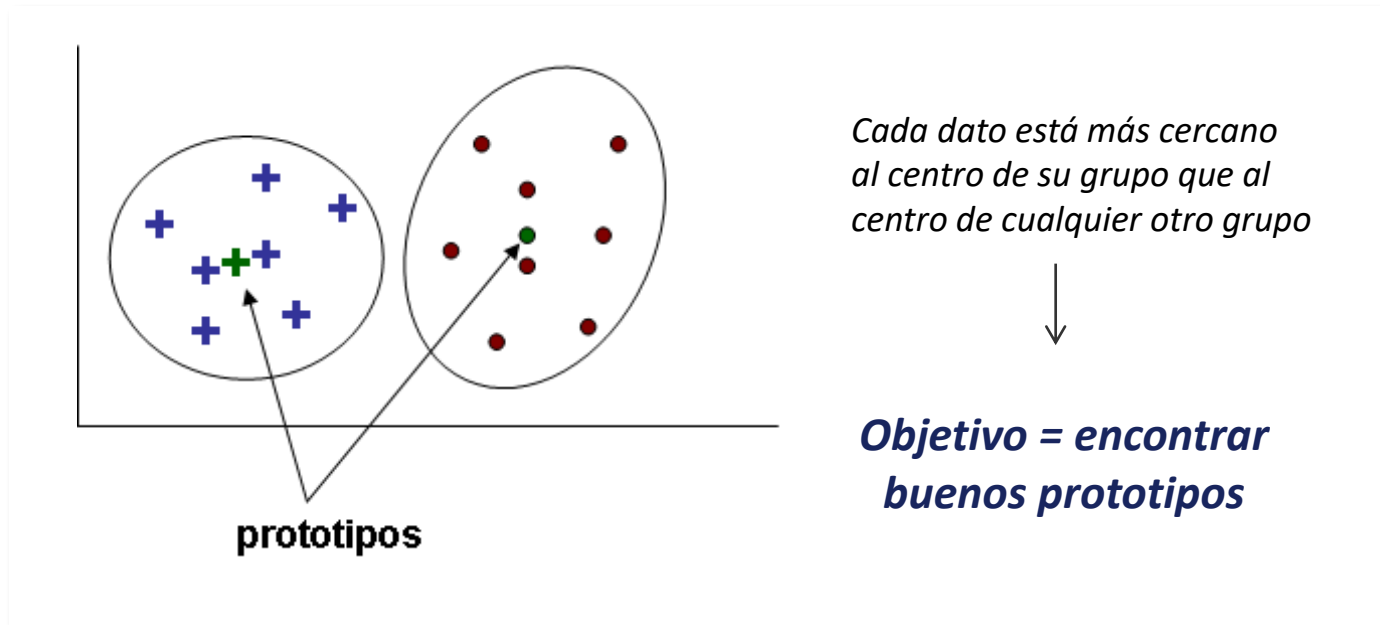
Las medidas de similitud juegan un rol fundamental en el diseño de algoritmos de agrupación.

Algunos algoritmos:

	Atributo 1	Atributo 2		Atributo d
Instancia 1				
Instancia 2				
Instancia n				



- a) Criterio de división: grupos al mismo nivel.
 - b) Separación de los grupos: en general encuentran grupos exclusivos.
 - c) Medida de similitud: la similitud se determina con medidas de distancia (como la Euclídea).
-
- El número de grupos debe ser especificado por el usuario.
 - Se utiliza un prototipo o centro para representar a los grupos.



- Se utiliza una función objetivo (que se busca optimizar) que mide la variación dentro de los grupos.

$$E = \sum_{i=1}^k \sum_{x \in G_i} \text{dist}(x, c_i) \quad \text{donde} \quad \begin{cases} x \in D = \{x_j\}_{j=1 \dots N} \\ G_i = \text{grupo asociado al prototipo } c_i, i = 1 \dots k \end{cases}$$

- Al minimizar esta función se intenta obtener grupos con una alta similitud intra-grupo y baja similitud inter-grupos (idea: obtener grupos tan compactos y separados como sea posible)

- Sea $D = \{x_j\}_{j=1\dots N}$ el conjunto de datos, el algoritmo busca K particiones disjuntas; cada una éstas están asociada a un grupo $G_i, i = 1..k$
- Cada grupo G_i tiene un representante c_i el cual es el centro del grupo = prototipo.
- El prototipo puede especificarse como la media de los datos asociados al grupo.
- Un dato se asigna al grupo G_i si él está más cercano al prototipo c_i que a cualquier otro prototipo.
- Se utiliza la distancia Euclídea como medida de similitud.

$$\Rightarrow E = \sum_{i=1}^k \sum_{x \in G_i} \|x - c_i\|^2$$

Algoritmo K-medias

Entrada = $\{D: \text{conjunto de datos}, K: \text{número de grupos}\}$

Comienzo_procedimiento

Seleccionar K datos de D como prototipos iniciales $c_i, i=1\dots K$

Repetir

Crear las particiones G_i asignando cada dato de D a su prototipo más cercano

Actualizar cada prototipo c_i como la media de los datos de la partición G_i

hasta alcanzar la convergencia

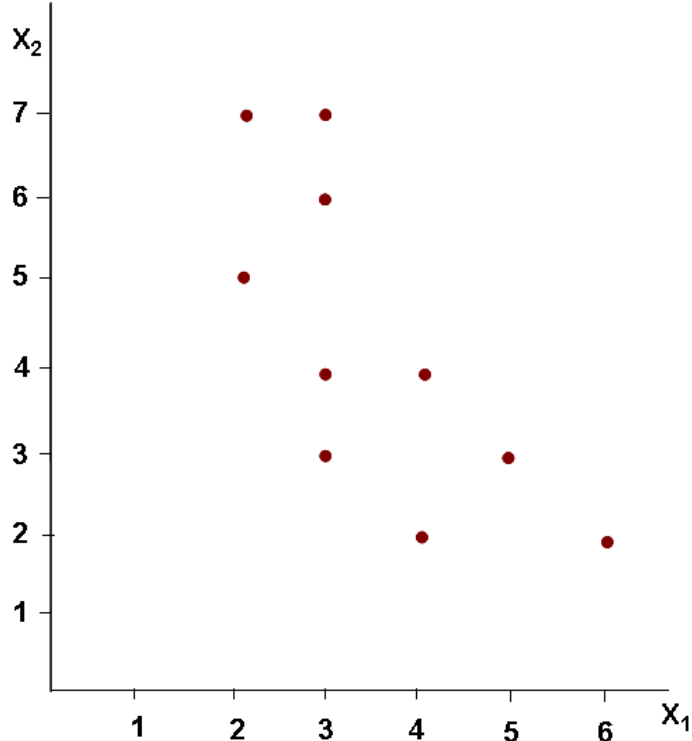
Fin-procedimiento

Salida = Conjunto de prototipos $c_i, i=1..K$

Conjunto de datos = $\{x_j\}_{j=1..10}$

$x_j = (x_{j1}, x_{j2}) = \text{valores de atributos}$

x_1	x_2
2	5
2	7
3	6
3	7
3	3
3	4
4	2
4	4
5	3
6	2



Documentos representados por un vector
utilizando frecuencias absolutas.

Comienzo_procedimiento

- Seleccionar 2 datos como centros iniciales:

$$c_1 = \begin{pmatrix} 3.00 \\ 3.00 \end{pmatrix} \quad c_2 = \begin{pmatrix} 4.00 \\ 2.00 \end{pmatrix}$$

- Primera iteración:

1er. paso. Determinar las particiones asignando cada dato a su centro más cercano.

dato	$d(x^i, C_1)$	$d(x^i, C_2)$
x^1	2.23	3.60
x^2	4.12	5.38
x^3	3.00	4.12
x^4	4.00	5.09
x^5	0.00	1.41
x^6	1.00	2.23
x^7	1.41	0.00
x^8	1.41	2.00
x^9	2.00	1.41
x^{10}	3.16	2.00

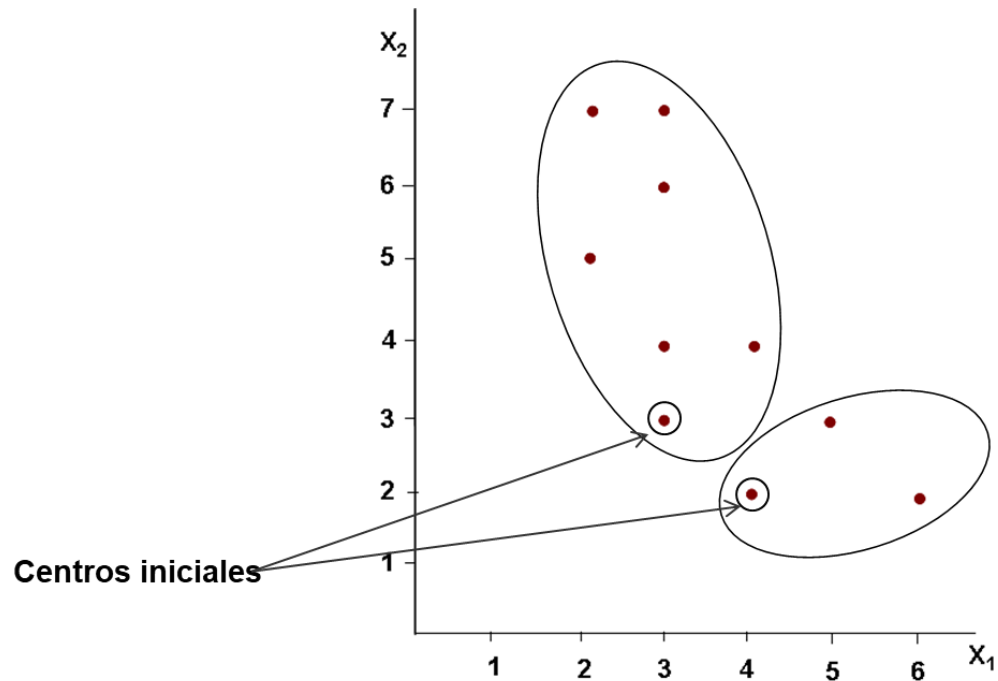


$$G_1 = \{ x^1, x^2, x^3, x^4, x^5, x^6, x^8 \}$$

$$G_2 = \{ x^7, x^9, x^{10} \}$$

↑ ↑
Distancia euclídea entre cada dato x^i
y los centros iniciales C_1 y C_2 .

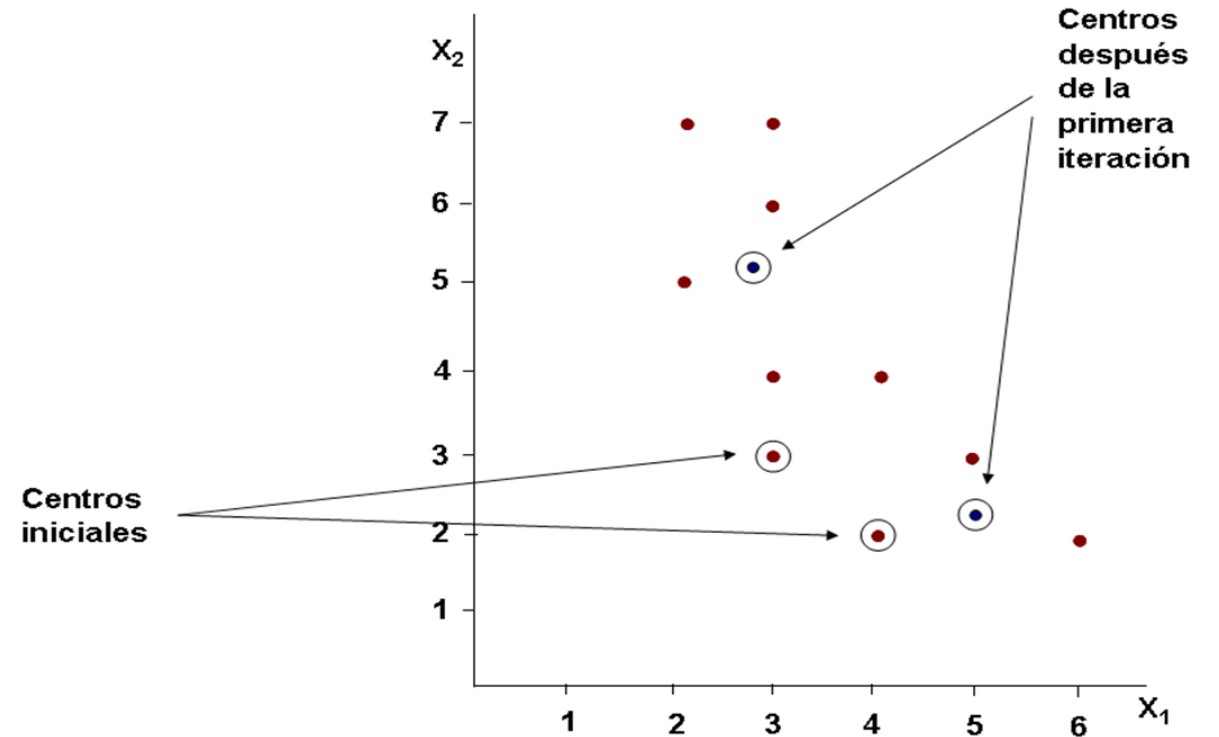
Particones obtenidas



2do. paso: actualizar cada prototipo C_i como la media de los datos de la partición G_i

$$c_1 = \frac{1}{7} \left\{ \begin{pmatrix} 2 \\ 5 \end{pmatrix} + \begin{pmatrix} 2 \\ 7 \end{pmatrix} + \begin{pmatrix} 3 \\ 6 \end{pmatrix} + \begin{pmatrix} 3 \\ 7 \end{pmatrix} + \begin{pmatrix} 3 \\ 3 \end{pmatrix} + \begin{pmatrix} 3 \\ 4 \end{pmatrix} + \begin{pmatrix} 4 \\ 4 \end{pmatrix} \right\} = \begin{pmatrix} 2.85 \\ 5.14 \end{pmatrix}$$

$$c_2 = \frac{1}{3} \left\{ \begin{pmatrix} 4 \\ 2 \end{pmatrix} + \begin{pmatrix} 5 \\ 3 \end{pmatrix} + \begin{pmatrix} 6 \\ 2 \end{pmatrix} \right\} = \begin{pmatrix} 5.00 \\ 2.33 \end{pmatrix}$$



Como los centros de la iteración 1 no son iguales a los centros iniciales, repetir.

- **Segunda iteración:**

$$c_1 = \begin{pmatrix} 2.83 \\ 5.50 \end{pmatrix}, c_2 = \begin{pmatrix} 4.50 \\ 2.50 \end{pmatrix}$$

- **Tercera iteración:**

$$c_1 = \begin{pmatrix} 2.60 \\ 5.80 \end{pmatrix}, c_2 = \begin{pmatrix} 4.40 \\ 2.80 \end{pmatrix}$$

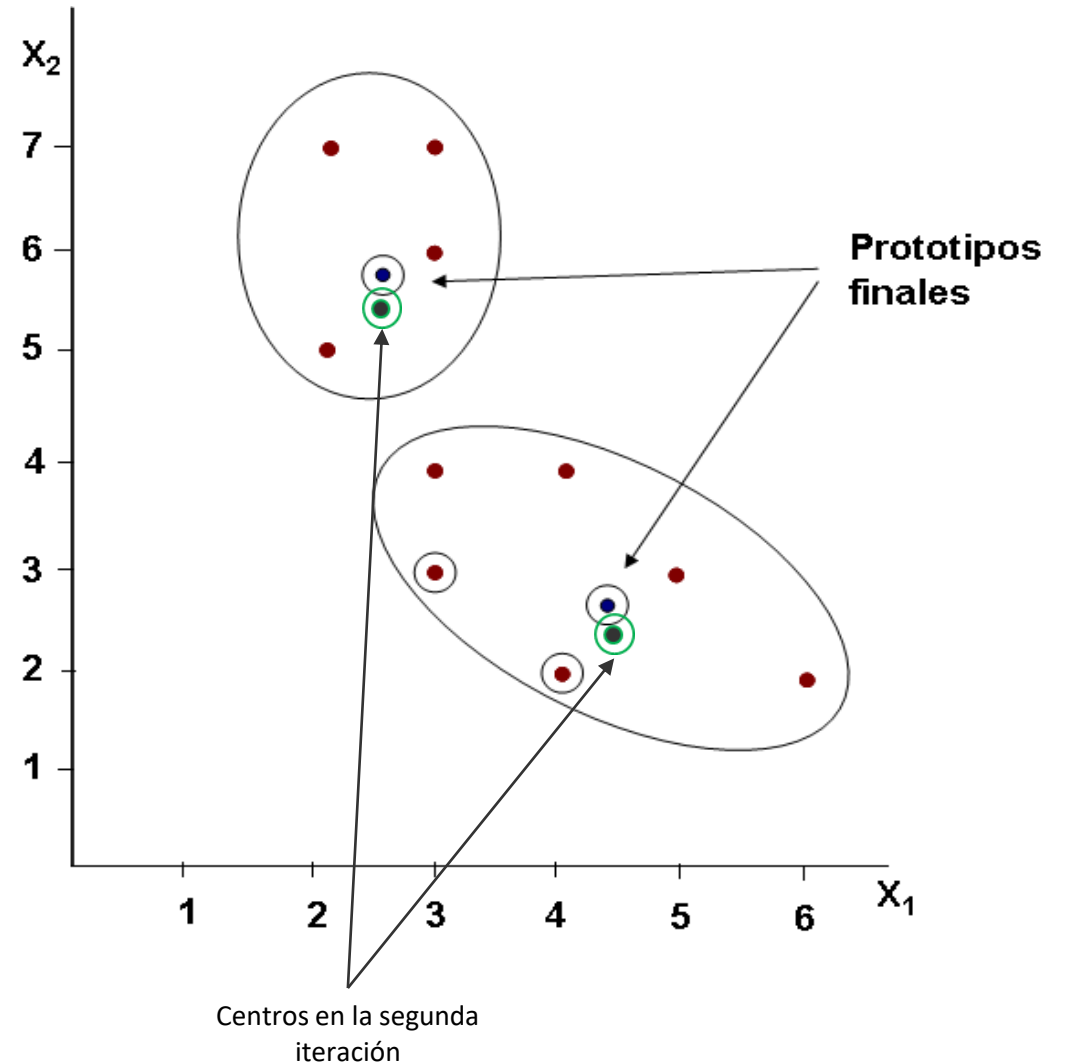


- **Cuarta iteración:**

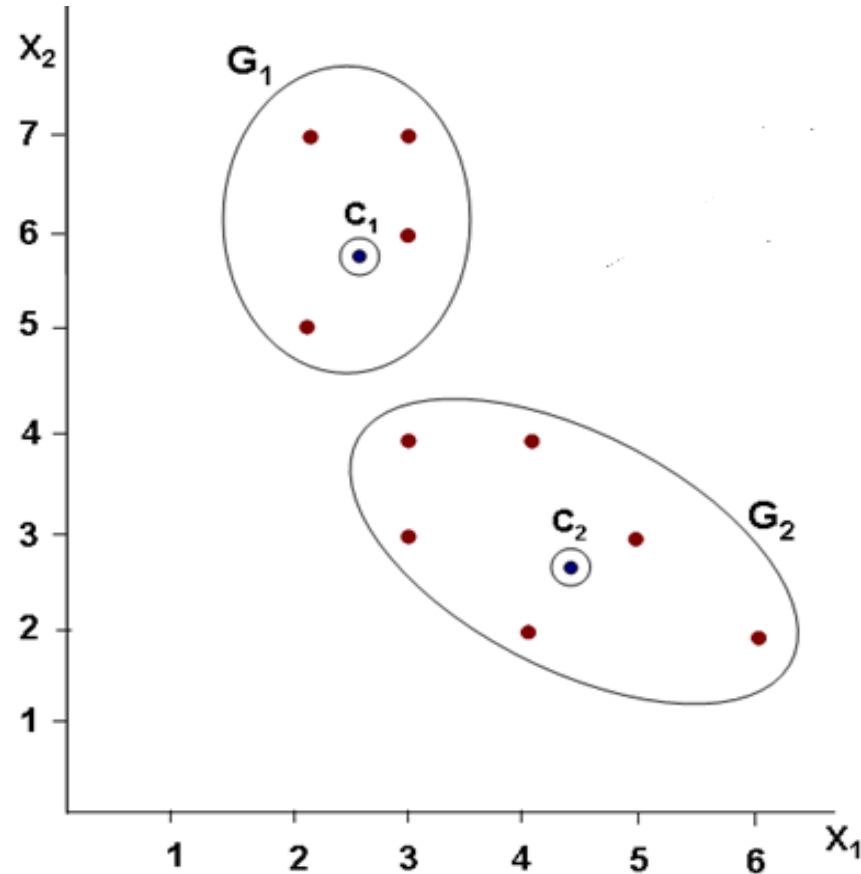
$$c_1 = \begin{pmatrix} 2.60 \\ 5.80 \end{pmatrix}, c_2 = \begin{pmatrix} 4.40 \\ 2.80 \end{pmatrix}$$



Convergencia



Agrupación final:



Una vez determinados los grupos, se pueden asignar etiquetas a estos utilizando conocimiento del dominio.

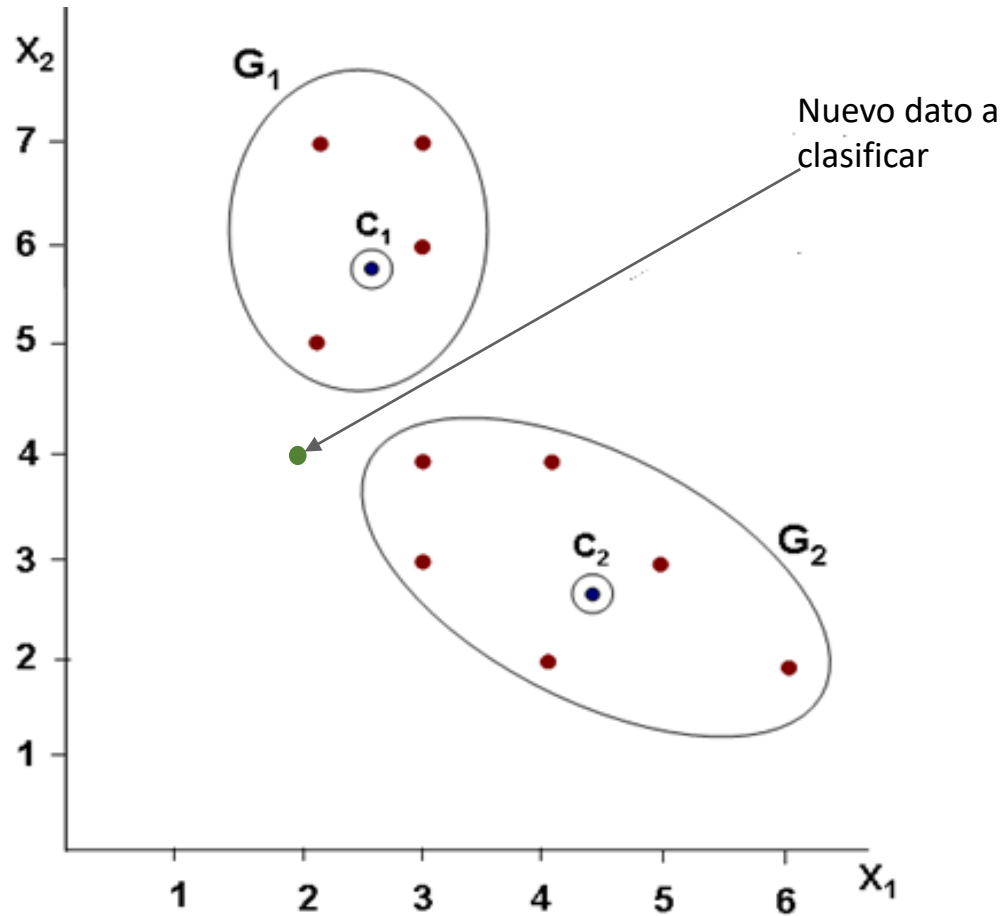
Esto se logra examinando las características de los objetos dentro de cada grupo (por ejemplo: G_1 es "Tópico A", G_2 es "Tópico B")

¿Cómo clasificar?



- ➡ Para clasificar un nuevo dato, se calcula la distancia de éste a cada centro o prototipo.
- ➡ Se asigna el nuevo dato al grupo con centro más cercano.

Si llega un nuevo documento nuevo a clasificar:



$$\mathbf{z} = \begin{pmatrix} 2 \\ 4 \end{pmatrix}$$

- Se calcula la distancia del nuevo dato a cada prototipo:

$$\begin{cases} \text{dist}(\mathbf{z}, \mathbf{c}_1) = 3.60 \\ \text{dist}(\mathbf{z}, \mathbf{c}_2) = 7.20 \end{cases}$$

- Se asigna el nuevo dato al grupo con centro más cercano

$\mathbf{z}$ más cercano a $\mathbf{c}_1 \Rightarrow \text{clase}(\mathbf{z}) = \text{Tópico A}$

❏ Características:

- En cada iteración se mejora la variación dentro de los grupos al calcular un nuevo prototipo.
- La convergencia puede ser obtenida con pocas iteraciones.
- Se pueden establecer otros criterios de parada (como el no. de iteraciones).
- A priori no se conoce el número de grupos (K).
- Los resultados pueden depender de las centros iniciales.
- No es adecuado para descubrir grupos no convexos.
- Es sensible a los *outliers* (cuando se utiliza la distancia euclídea).



Algoritmo	K-medias
Tarea	Agrupación
Parámetros	Número de grupos (K), medida de distancia o de similitud.
Características	Requiere de atributos numéricos. Si hay atributos nominales en el conjunto de datos estos deben ser transformados a números mediante un proceso de numerización. Es conveniente realizar un escalado de estos atributos (normalización o estandarización). Importante definir el valor de K (ajustarlo al conjunto de datos disponible).

- ❑ **Métodos intrínsecos:** determina la calidad de la agrupación utilizando medidas que cuantifican la separabilidad y la compacidad de los grupos.

La compacidad está relacionada con la distancia intra grupo (*intra-clúster*) y mide su nivel de cohesión interna, es decir, qué tan cercanos están los datos dentro de un grupo. La separabilidad mide la distancia inter grupos (*inter-clúster*), es decir, que tan separados están los grupos entre sí.

Estas métricas se pueden utilizar para determinar el número de grupos para el conjunto de datos.

Algunas medidas que pueden utilizarse son:

Suma de las distancias al cuadrado para cada grupo. Se utiliza para cuantificar la cohesión interna o compacidad de los grupos, en general para algoritmos basados en prototipos como K-medias. Esto se logra calculando la varianza intra grupo como la suma de las distancias al cuadrado de cada dato x a su prototipo:

$$E = \sum_{i=1}^k \sum_{x \in G_i} \|x - c_i\|^2$$

Donde

$$x \in D = \{x_i\}_{i=1 \dots N}$$

$$G_j = \text{grupo asociado al prototipo } c_j, j = 1 \dots k$$

También se conoce como *inercia*.
Cuanto menor sea el valor, mejor será la agrupación.

Coeficiente de silueta. Permite cuantificar la cohesión interna y la separabilidad de los grupos. Para cada dato $x \in D = \{x_i\}_{i=1\dots N}$ se calcula:

$$\text{coeficiente de silueta}(x) = cs(x) = \frac{d_{inter}(x) - d_{intra}(x)}{\max\{d_{inter}(x), d_{intra}(x)\}}$$

Donde

d_{inter} = distancia promedio mínima de x a los puntos del grupo más cercano.

Este valor indica el grado de separación del objeto x al resto de los grupos. Mientras más grande, más separado

d_{intra} = distancia promedio de x a los puntos dentro de su grupo.

Para determinar la calidad de la agrupación se utiliza entonces el promedio de estos valores sobre el conjunto de datos:

$$\text{coeficiente de silueta (agrupación)} = cs_{agrupación} = \frac{1}{N} \sum_{i=1}^N cs(x_i)$$

Este promedio está limitado al rango [-1 y 1]. Cuando el valor está cercano a 1, sugiere grupos compactos y separados. Un valor cercano a -1 nos da idea de una agrupación incorrecta (mala asignación a un grupo), ya que es evidencia de que los datos están más cercanos a los puntos de otros grupos. Los valores en torno a cero indican que los grupos se solapan (datos en el límite entre grupos).

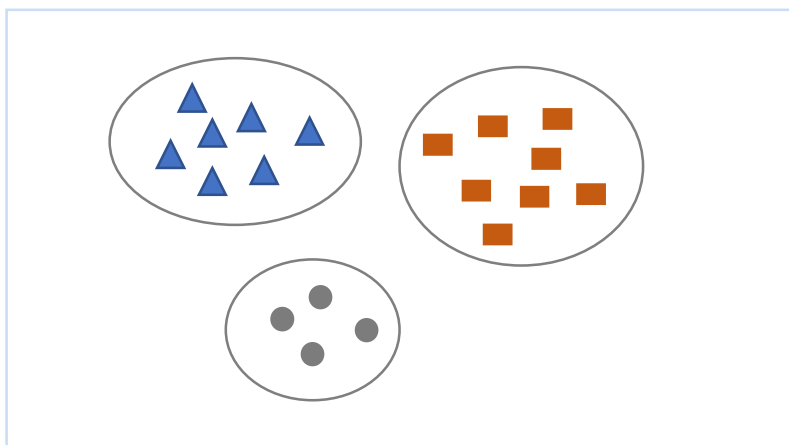
❑ **Métodos extrínsecos:** utiliza el conocimiento experto de manera implícita, a través de un conjunto de datos etiquetados. Estas etiquetas representan la “verdad o realidad fundamental” (*ground truth*) del problema.

Las etiquetas indican las clases a la que pertenecen los objetos del conjunto de datos y son utilizadas de manera explícita en métricas que determinan el grado al cual datos que son de la misma clase están incluidos en el mismo grupo, es decir, el “acuerdo” en las asignaciones de los datos a los grupos y el *ground truth*.

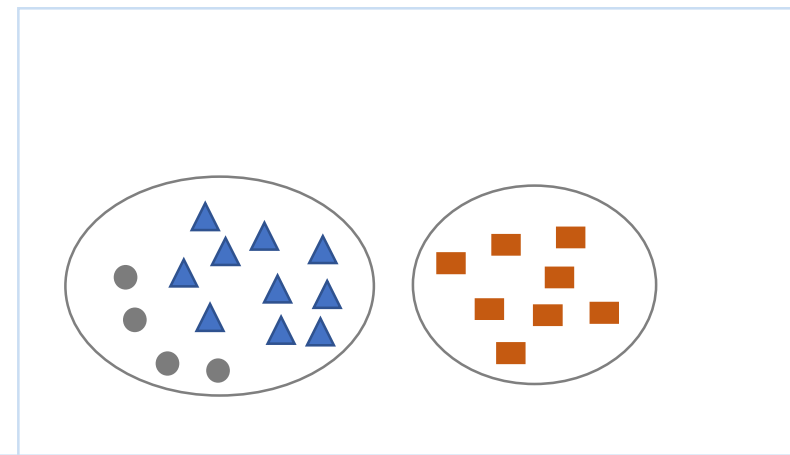
Ejemplo:

$$V_{score} = 2 \times \frac{\text{homogeneidad} \times \text{completitud}}{\text{homogeneidad} + \text{completitud}}$$

Esta medida toma valores en el intervalo [0, 1]. Valores cercanos a 1 indican que tanto la homogeneidad como la completitud están también próximos a 1, significando una buena agrupación.



La homogeneidad indica si los grupos solo contienen datos de una sola clase. Representa la “pureza” de la agrupación.



La completitud nos dice si todos los datos de una clase fueron asignados al mismo grupo.

Aspectos de implementación de k- means

A) Inicialización del algoritmo → Importancia de los centros iniciales

Ejemplo: Conjunto de datos

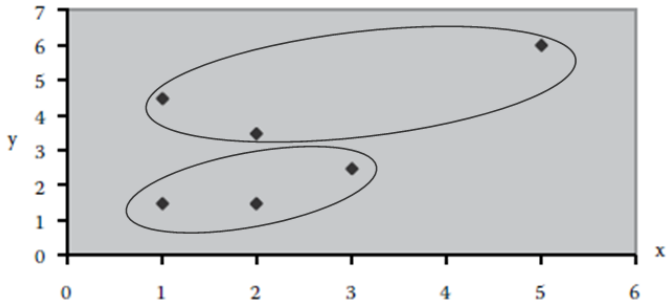
No.	X Numeric	Y Numeric
1	1.0	1.5
2	1.0	4.5
3	2.0	1.5
4	2.0	3.5
5	3.0	2.5
6	5.0	6.0

Partiendo de diferentes condiciones iniciales:

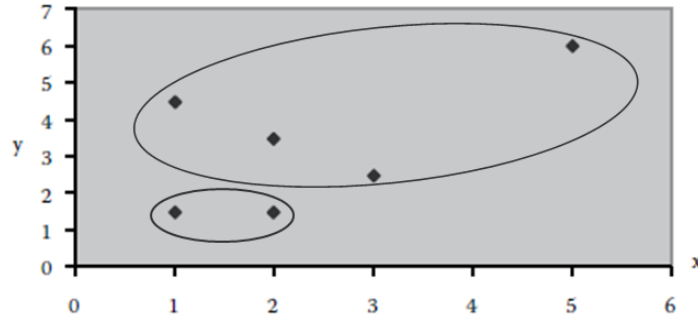


Agrupación	Centros de los grupos	Datos de los grupos	Error
1	(2.67,4.67)	2, 4, 6	14.50
	(2.00,1.83)	1, 3, 5	
2	(1.5,1.5)	1, 3	15.94
	(2.75,4.125)	2, 4, 5, 6	
3	(1.8,2.7)	1, 2, 3, 4, 5	9.60
	(5,6)	6	

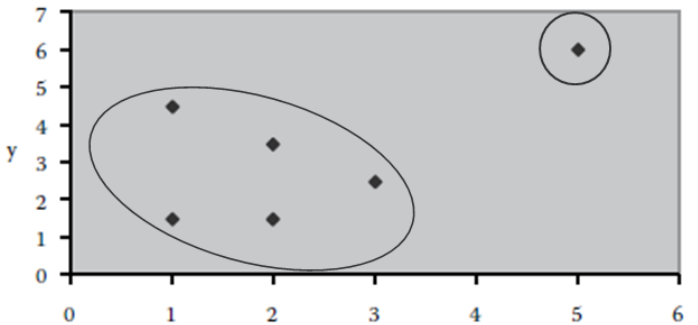
Agrupación 1



Agrupación 2



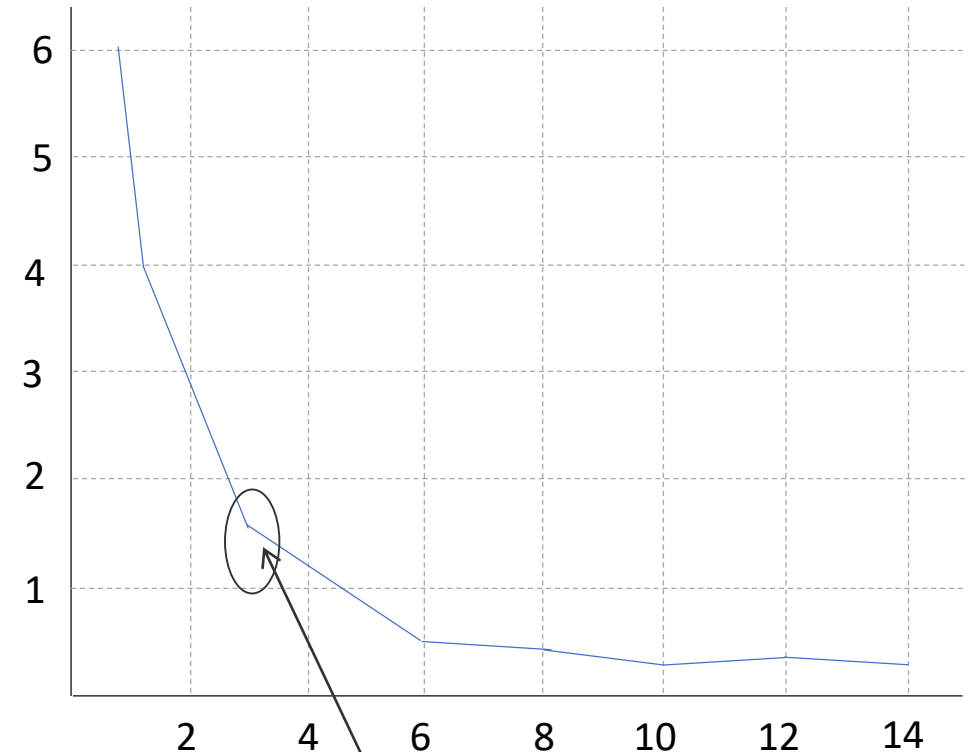
Agrupación 3



B) ¿Cómo determinar el número de grupos?

Método del codo

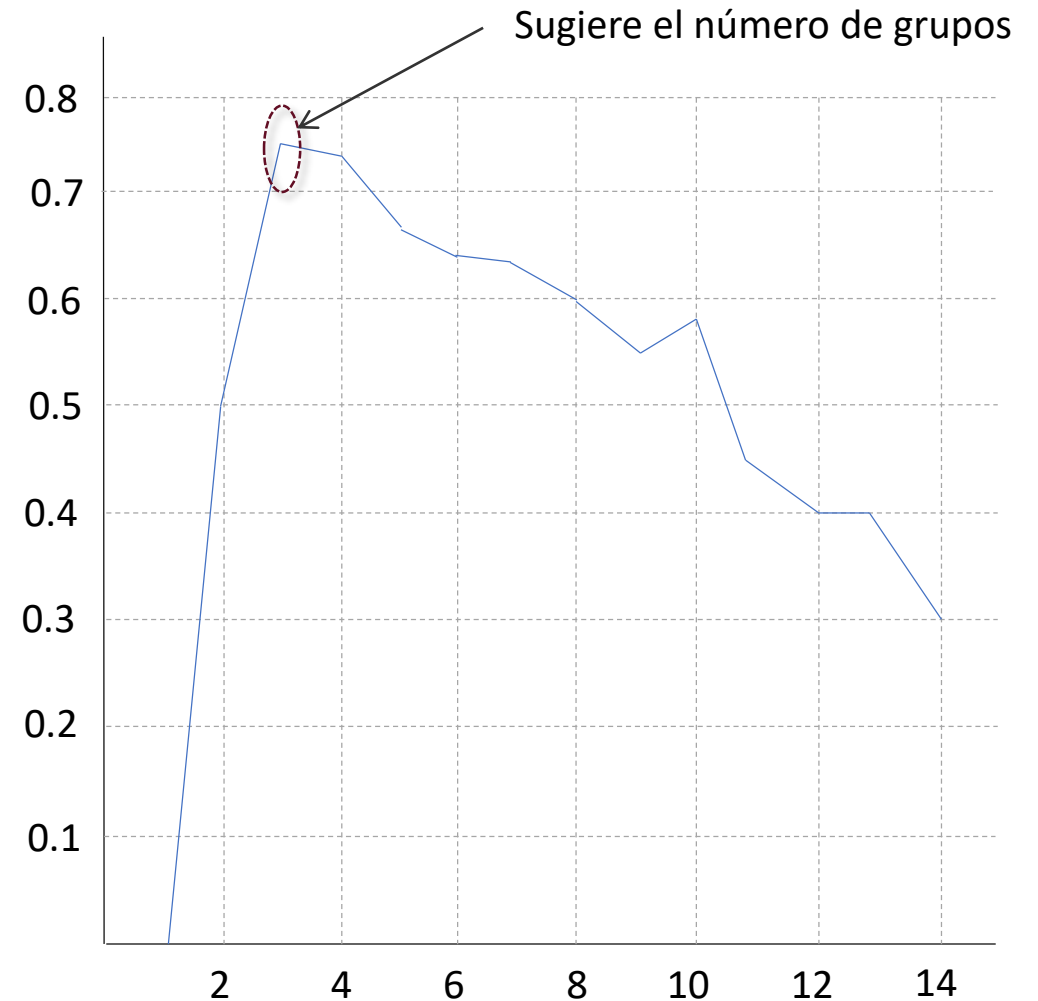
- Este método se basa en la métrica de inercia o suma de las distancias al cuadrado (SSE) para determinar el número de grupos.
- Para varios valores de K se calcula el valor de esta métrica para la agrupación resultante.
- Luego se construye una gráfica como se muestra a continuación:
- En la gráfica se observa un primer cambio brusco en la evolución de SSE, que asemeja un “codo”. Este punto puede dar información del número de grupos para ese conjunto de datos. En este caso, sería $K = 3$.
- Es importante resaltar que este es un método heurístico, lo cual quiere decir que en muchos casos puede resultar útil y acertar en el número de grupos adecuado para el problema, pero no siempre será así.



Sugiere el número de grupos

Método de la silueta

- Este método se basa en la métrica de silueta de la agrupación para determinar el número de grupos.
- Para varios valores de K se calcula el valor de esta métrica para la agrupación resultante.
- Luego se construye una gráfica como se muestra a continuación:
- En la gráfica se observa cómo evoluciona el valor del coeficiente de silueta a medida que se aumenta el número de grupos. Con $K = 3$ se obtiene el mejor valor de esta métrica, lo que sugiere que este debería ser el número de grupos a utilizar para este conjunto de datos.
- Es importante resaltar que este es un método heurístico, lo cual quiere decir que en muchos casos puede resultar útil y acertar en el número de grupos adecuado para el problema, pero no siempre será así.



```
from sklearn.cluster import KMeans
```

Parameters:

- n_clusters : int, default=8**
The number of clusters to form as well as the number of centroids to generate.
- init : {'k-means++', 'random'} or ndarray of shape (n_clusters, n_features), default='k-means++'**
Method for initialization, defaults to 'k-means++':
 - 'k-means++': selects initial cluster centers for k-mean clustering in a smart way to speed up convergence. See section Notes in k_init for more details.
 - 'random': choose k observations (rows) at random from data for the initial centroids.If an ndarray is passed, it should be of shape (n_clusters, n_features) and gives the initial centers.
- n_init : int, default=10**
Number of time the k-means algorithm will be run with different centroid seeds. The final results will be the best output of n_init consecutive runs in terms of inertia.
- max_iter : int, default=300**
Maximum number of iterations of the k-means algorithm for a single run.

Realiza varias ejecuciones del algoritmo con diferentes condiciones iniciales. Para determinar la mejor agrupación utiliza el factor de inercia (= sumas e las distancias al cuadrado en los grupos)

$$E = \sum_{i=1}^k \sum_{x \in G_i} \|x - c_i\|^2$$

Una vez determinada la agrupación se puede asignar un grupo a una nueva instancia:

```
In [20]: from sklearn.cluster import KMeans
import numpy as np
X = np.array([[1, 2], [1, 4], [1, 0], [10, 2], [10, 4], [10, 0]])
kmeans = KMeans(n_clusters=2, random_state=0).fit(X)
kmeans.predict([[0, 0], [12, 3]])
```

```
Out[20]: array([1, 0])
```

Interpretación de la agrupación

Ejemplo 1: una empresa de venta de productos comestibles desea conocer los patrones de compra de sus clientes para ofrecer un servicio más personalizado¹.

Channel	Region	Fresh	Milk	Grocery	Frozen	Detergents_Paper	Delicassen
2	3	12669	9656	7561	214	2674	1338
2	3	7057	9810	9568	1762	3293	1776
2	3	6353	8808	7684	2405	3516	7844
1	3	13265	1196	4221	6404	507	1788
2	3	22615	5410	7198	3915	1777	5185
2	3	9413	8259	5126	666	1795	1451
2	3	12126	3199	6975	480	3140	545
2	3	7579	4956	9426	1669	3321	2566
1	3	5963	3648	6192	425	1716	750
2	3	6006	11093	18881	1159	7425	2098
2	3	3366	5403	12974	4400	5977	1744
2	3	13146	1124	4523	1420	549	497
2	3	31714	12319	11757	287	3881	2931
2	3	21217	6208	14982	3095	6707	602
2	3	24653	9465	12091	294	5058	2168
1	3	10253	1114	3821	397	964	412
2	3	1020	8816	12121	134	4508	1080
1	3	5876	6157	2933	839	370	4478
2	3	18601	6327	10099	2205	2767	3181
1	3	7780	2495	9464	669	2518	501
2	3	17546	4519	4602	1066	2259	2124
1	3	5567	871	2010	3383	375	569
1	3	31276	1917	4469	9408	2381	4334
2	3	26373	36423	22019	5154	4337	16523
2	3	22647	9776	13792	2915	4482	5778

Descripción de los atributos:

- 1) Fresh: gasto anual (m.u.) en productos frescos (numérico)
- 2) Milk: el gasto anual (m.u.) en los productos lácteos (numérico)
- 3) Grocery: el gasto anual (m.u.) en los productos comestibles (numérico)
- 4) Frozen: gasto anual (m.u.) en productos congelados (numérico)
- 5) Detergents: gasto anual (m.u.) en detergentes y productos de papel (numérico)
- 6) Delicatessen: gasto anual (m.u.) y en productos delicatessen (numérico)
- 7) Channel: Canal - 1: Horeca (Hotel / Restaurante / Café) - 2: canal de venta al menor (nominal)
- 8) Region: Región 1:Lisboa, 2:Oporto, 3:Otros (nominal)

¹Conjunto de datos: <https://archive.ics.uci.edu/ml/datasets/wholesale+customers>

Con 3 grupos:

```
Number of iterations: 9
Within cluster sum of squared errors: 84.3999157285306
Missing values globally replaced with mean/mode

Cluster centroids:

Attribute          Full Data          Cluster#
                   (440)          0          1          2
                   (440)          (220)          (138)          (82)
=====
Channel              1              1              2              1
Region              3              3              3              1
Fresh             12000.2977  14224.6364  8892.9638  11261.9756
Milk               5796.2659   3501.3091  10888.1812  3384.1463
Grocery            7951.2773   3934.8045  16606.2609  4161.4756
Frozen             3071.9318   3894.3318   1565.4565  3400.7805
Detergents_Paper   2881.4932   779.5591   7410.4565   898.9146
Delicassen         1524.8705   1523.7182   1766.7971  1120.8171

Time taken to build model (full training data) : 0.02 seconds

=== Model and evaluation on training set ===

Clustered Instances

0      220 ( 50%)
1      138 ( 31%)
2       82 ( 19%)
```

Posible interpretación de los grupos:

- Grupo 0: son los clientes que gastan más en productos frescos, y están asociados al canal HORECA. Gastan poco en productos de limpieza. Y están asociados a otras regiones.
- Grupo 1: Son minoristas y los que más gastan en productos lácteos y comestibles, así como en productos de limpieza. También son los que más gastan en delicatessen y menos en congelados.
- Grupo 2: Asociados al canal HORECA, representan un 19% de los clientes. Son también grandes compradores de productos frescos de la región de Lisboa. Gastan poco en productos de limpieza.

Ejemplo 2. Detección de Anomalías en Tráfico de Redes de Computadores*

Objetivos: encontrar comportamientos inusuales en el tráfico de red con el fin de identificar patrones de ciberataques, aumentar la capacidad de respuesta a riesgos de seguridad, bloquear intrusos a la red y mejorar la protección de datos confidenciales.

a) Recolección de los datos.

- Datos sobre la actividad de tráfico de red, capturada en los firewalls de los clientes, recopilados en un día de actividad.

b) Preparación de los datos.

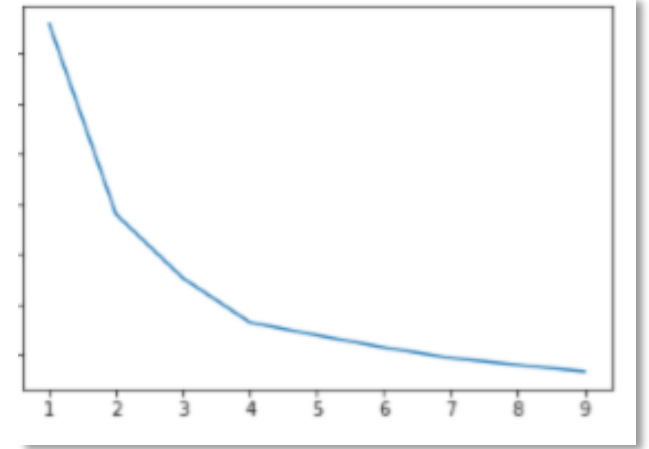
- Eliminación de registros con ausencias de datos básicos (por ejemplo, el trafico de red siempre debe estar asociado a una acción, duración y numero de paquetes de la sesión).
- Eliminación de atributos no informativos, como policyid y user, ya que son identificadores que no aportan al análisis.
- Eliminación de variables altamente correlacionadas (superior al 90%), quedando para el análisis: srcport, dest_port, action, dstcountry, duration, packets, proto, rcvdpkt, transport, num_srcip y num_dstip.
- Transformación de variables nominales a numéricas.
- Normalización de todas las variables.

Campos	Descripción
_time	Tiempo del evento
subtype	Describe el tipo de log
level	Nivel del log
scrip	Ip fuente de tráfico
srcport	Puerto fuente de tráfico
srcintf	Interfaz de firewall en el que se inicia la conexión
Dest_ip	Ip destino de la conexión
Dest_interface	Interfaz destino de la conexión
Dest_port	Puerto destino de la conexión
action	Acción que toma el firewall para la conexión
dstcountry	País destino de la conexión
srccountry	País origen de la conexión
duration	Duración de la sesión
Ftnt_action	Acción que toma el UTM del firewall
packets	Paquetes vistos dentro de la sesión de la conexión
Packets_out	Paquetes que salen de la conexión
policyid	Política por la cual se permite la conexión
policytype	Tipo de política que está permitiendo la conexión
proto	Protocolo de conexión
rcvbyte	Bytes recibidos dentro de la conexión
sentbytes	Bytes enviados dentro de la conexión
service	Servicio que esta como destino de la conexión (http, https, ssh, rdp, smtp)
transip	Ip con la cual se está haciendo NAT para publicar el servicio
transport	Protocolo de transporte
user	Autenticación con la cual el firewall está permitiendo el servicio a un usuario

* Alejandro Agudelo, Stanley Melo, Wilman Rincón

c) Estimación de modelo:

- Se aplicó un algoritmo de agrupación (K-medias).
- Las pruebas realizadas arrojaron 4 grupos (utilizando el método de elbow).



d) Interpretación de los grupos:

Grupo 1: tráfico principalmente HTTP y HTTPS, entre IPs internas e IPs de cuatro países, las cuales no tiene comportamiento malicioso. El tráfico es permitido y se puede identificar comunicaciones dentro de la operación normal de un tráfico de la organización.

Grupo 2: tráfico principalmente UDP a DNS y NTP, esto dentro de los servicios normales de la operación de una organización, siempre hacia las mismas IPs, las cuales presentan un comportamiento normal, por lo tanto, no se consideraron un tráfico malicioso.

Grupo 3: tráfico bloqueado y tráfico permitido, el cual va hacia IPs de diferentes países, este tráfico debe ser analizado más a fondo por los analistas de seguridad, y se pueden considerar anomalías en el tráfico de red de los datos analizados.

Grupo 4: tráfico entre redes internas, en el cual se tiene muy poco número de paquetes transmitidos, y muy poco tiempo de conexión, todo este tráfico entre puertos muy altos, no se considera así un tráfico a ser analizado.

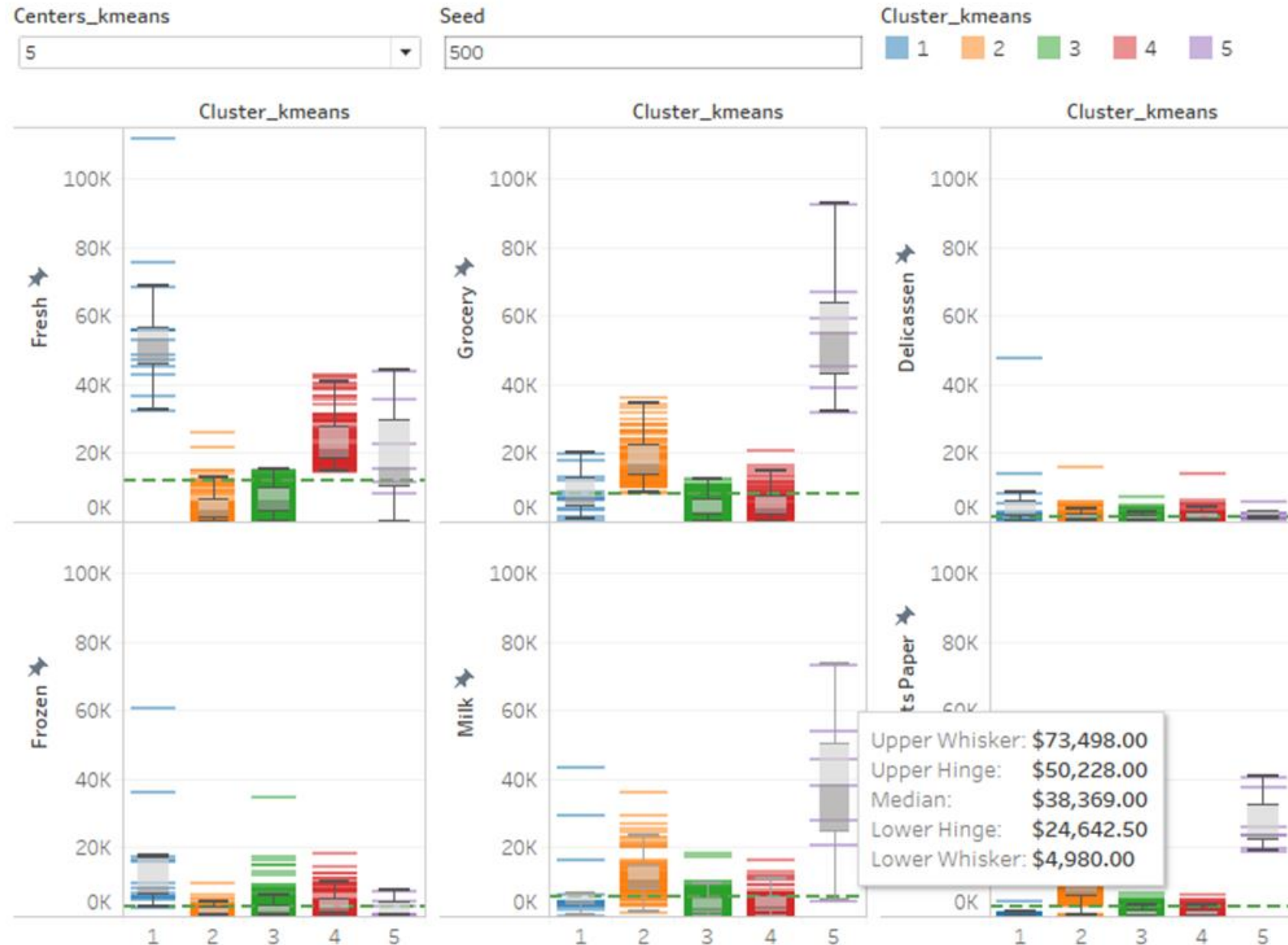
Ejemplo 3. Uno de los problemas al que se enfrentan los negocios del mercado de consumidores y de sectores empresariales es el abandono de los clientes. La solución tradicional pasa por predecir cuáles clientes exhiben una alta propensión a abandonar y abordar sus necesidades a través de un servicio de asistencia personalizada, campañas de marketing o mediante la aplicación de exenciones especiales. Un interesante aporte aquí es el análisis con técnicas de machine learning. Las empresas recopilan datos exhaustivos e históricos sobre sus clientes, y es posible utilizarlos para determinar modelos descriptivos que permitan caracterizar los clientes con intención de renuncia a los servicios contratados con la empresa.

Al aplicar el algoritmo K-medias, sobre datos de una de estas empresas, se obtuvieron los cuatro grupos que se muestran a continuación:

	Prototipos			
Variable	Grupo 1	Grupo 2	Grupo 3	Grupo 4
Score crediticio	653.95	641.27	650.16	653.88
Género	M	M	M	F
Edad	38.81	41.27	38.79	37.33
Balance	112461.41	77271.23	35859.12	75954.98
Tarjeta de crédito	1	1	0	1
Miembro activo	1	0	1	0

Junto con los expertos se determinó que el grupo 3 corresponde a clientes que han abandonado la empresa, mientras que el resto se relaciona con aquellos que todavía permanecen en esta. A partir del prototipo ¿cuáles reglas se pueden derivar para caracterizar estos dos patrones de comportamiento?

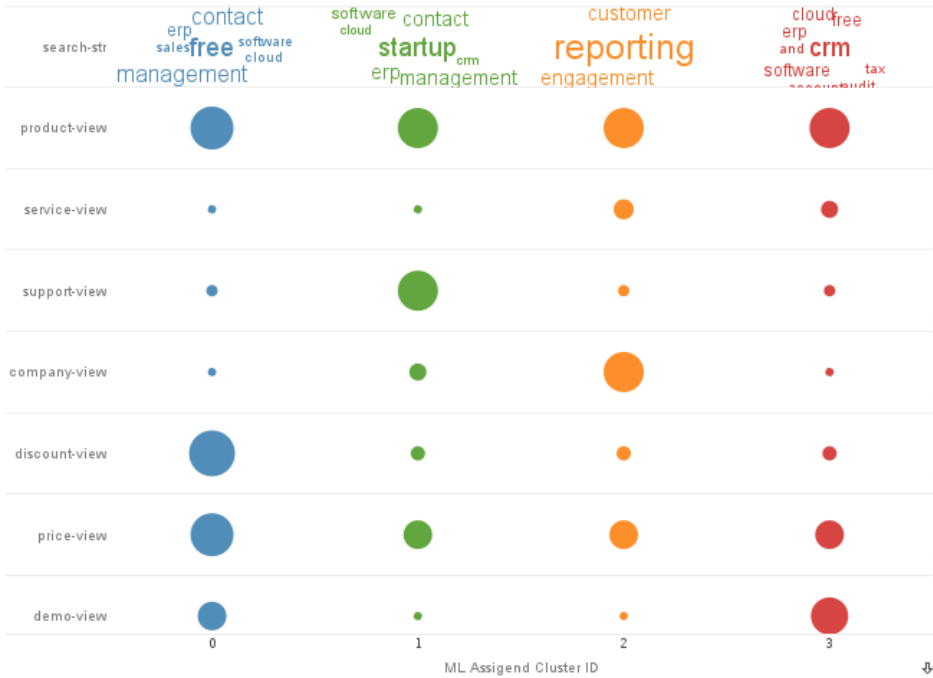
❏ Análisis descriptivo sobre la agrupación





ML Generated Visitor Clusters - with Keyword Concentration

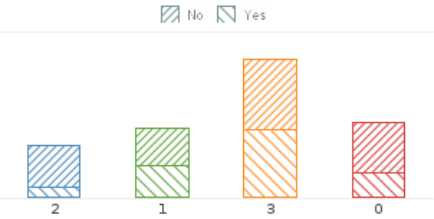
Avg time spent in each section of website shown by circle size



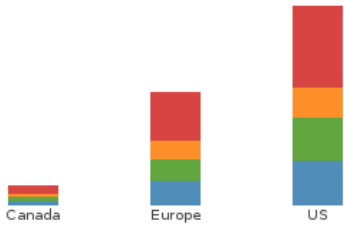
Website Sections to Show in Chart

- ☒ company-view
 ☒ demo-view
 ☒ discount-view
 ☒ price-view
 ☒ product-view
 ☒ search-str
- ☒ service-view
 ☒ support-view
 ☐ zone

Registered Visitors by Cluster



of Visitors by Region & Cluster

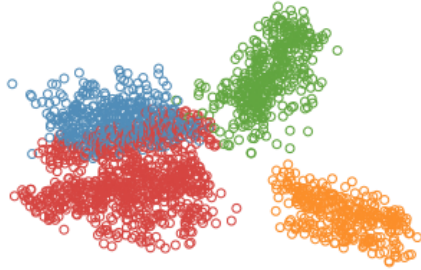


Limit Plotted Clusters

- ☒ 0
 ☐ 1
 ☐ 2
 ☐ 3
 + Others...

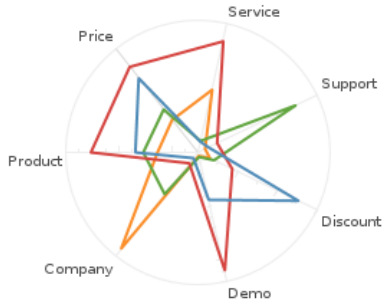
Cluster Distribution with Dimension Reduction

ML reduced dimensions to 2 for visualization. This is for a general sense of distribution, not a literal plotting of raw data points



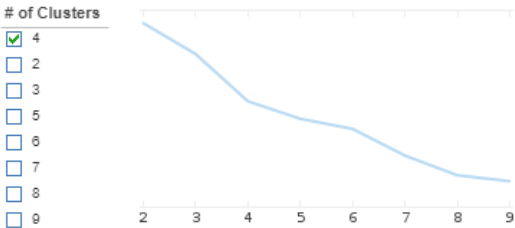
Total Time Spent in Website Sections

By Cluster



Specify # of Clusters for ML

Experiment with the # of clusters. Use the curve to find the "elbow" where the slope changes, which is potentially a good # of clusters.



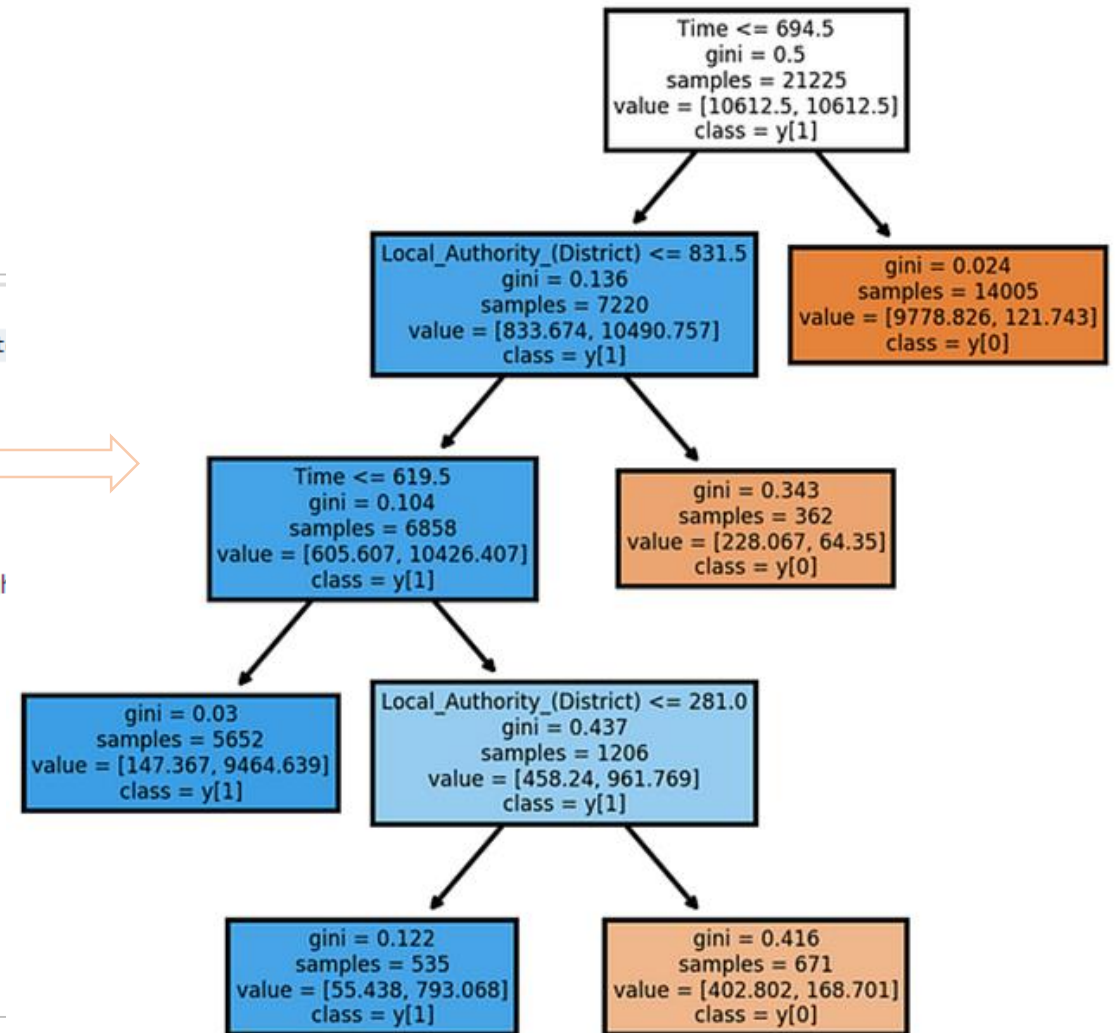
❏ Aplicación de un algoritmo interpretable.

Se asigna una etiqueta a cada dato, que representa el grupo al que pertenece.

En scikit-learn (K-means)

Attributes:

- cluster_centers_ :** *ndarray of shape (n_clusters, n_features)*
Coordinates of cluster centers. If the algorithm stops before fully converging (see `tol` and `max_iter`) will not be consistent with `labels_`.
- labels_ :** *ndarray of shape (n_samples,)*
Labels of each point
- inertia_ :** *float*
Sum of squared distances of samples to their closest cluster center, weighted by the sample weight provided.
- n_iter_ :** *int*
Number of iterations run.
- n_features_in_ :** *int*
Number of features seen during fit.
New in version 0.24.
- feature_names_in_ :** *ndarray of shape (n_features_in_,)*
Names of features seen during fit. Defined only when `x` has feature names that are all strings.



Interpretable Clustering

<https://towardsdatascience.com/interpretable-clustering-39b120f95a45>

Introduction to Interpretable Clustering

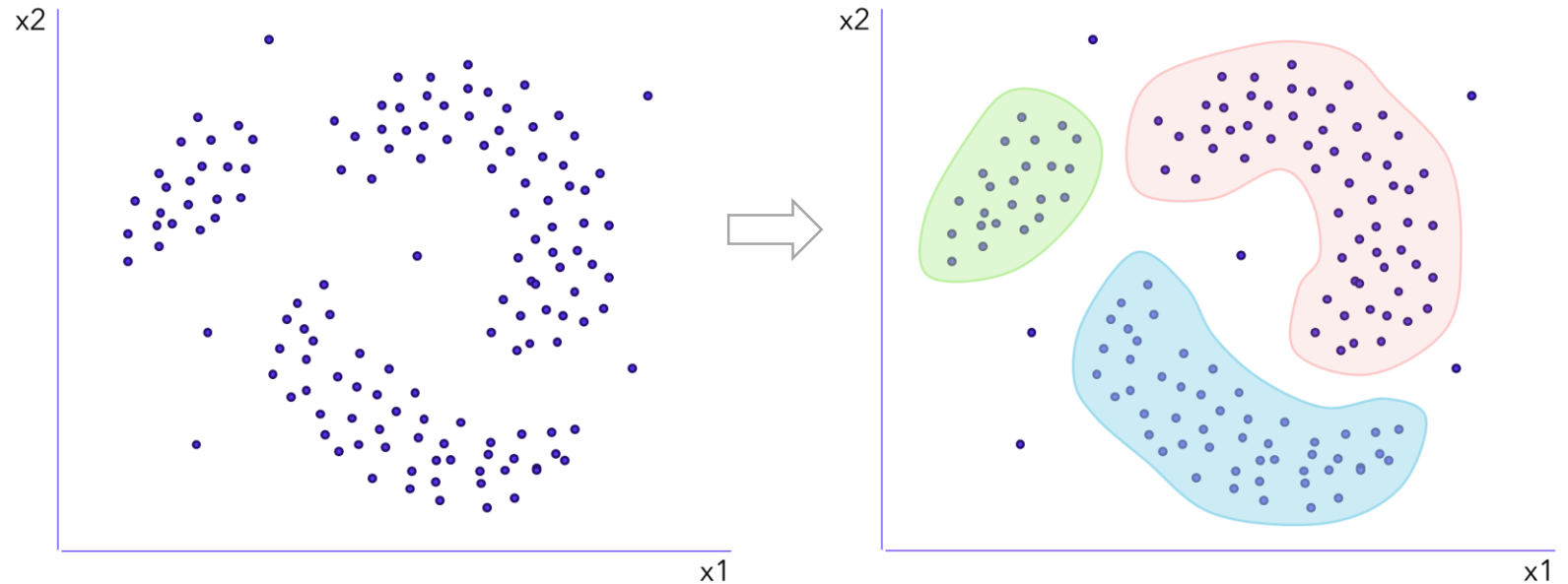
<https://towardsdatascience.com/introduction-to-interpretable-clustering-d0e07fbd2c99>

Otros algoritmos de agrupación

❑ Agrupación basada en densidad

- a) Criterio de división: grupos al mismo nivel.
- b) Separación de los grupos: se encuentran grupos exclusivos.
- c) Medida de similitud: basada en la conectividad (contigüidad)

- Un grupo es una región densa de objetos, que se separa de otros grupos por zonas de baja densidad.
- Los grupos pueden tener formas irregulares.
- Para construir los grupos se determina la densidad en el entorno de cada dato (la densidad de un objeto se mide por el número de objetos cercanos a él en una vecindad).



Ejemplo: BDSCAN (Density-based spatial clustering of applications with noise)

- Para determinar la densidad en torno a un objeto se utilizan dos parámetros (especificados por el usuario).

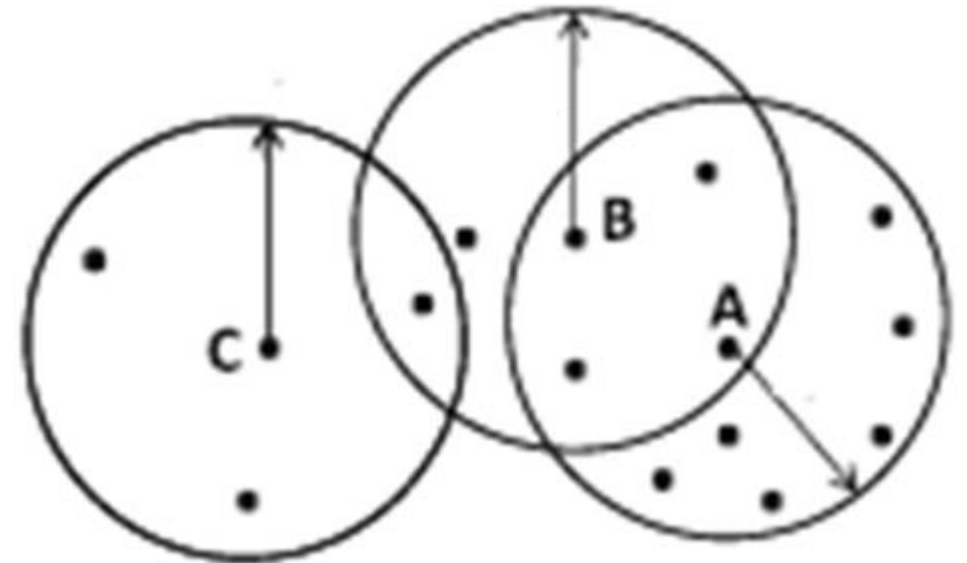
- ☞ Radio de la vecindad (épsilon, EPS)
- ☞ Número de objetos en la vecindad (*minObj*)

- Con estos parámetros se identifican en el conjunto de datos:

- ☞ **Objetos núcleo:** datos que en su vecindad tienen al menos *minObj* (*min_points/min_samples*) como vecinos (A).
- ☞ **Objetos borde:** datos que no tienen en su vecindad *minObj* (*min_points/min_samples*), pero están en la vecindad de un objeto núcleo (B).
- ☞ **Objetos extremos:** no cumplen con alguna de las condiciones anteriores (C).



Con *minObj* = 10



- Idea: identificar todos los objetos núcleo con respecto a ϵ y $minObj$. Estos objetos son los pilares de las regiones densas.
- La tarea de la agrupación: usar estos objetos y sus vecindades para formar las regiones densas = grupos.

Algoritmo DBSCAN

Entrada = {D: conjunto de datos, ϵ : tamaño o radio de la vecindad, $minObj$: número mínimo de datos en la vecindad r }

Comienzo_procedimiento

Determinar los datos núcleo, datos borde y outliers

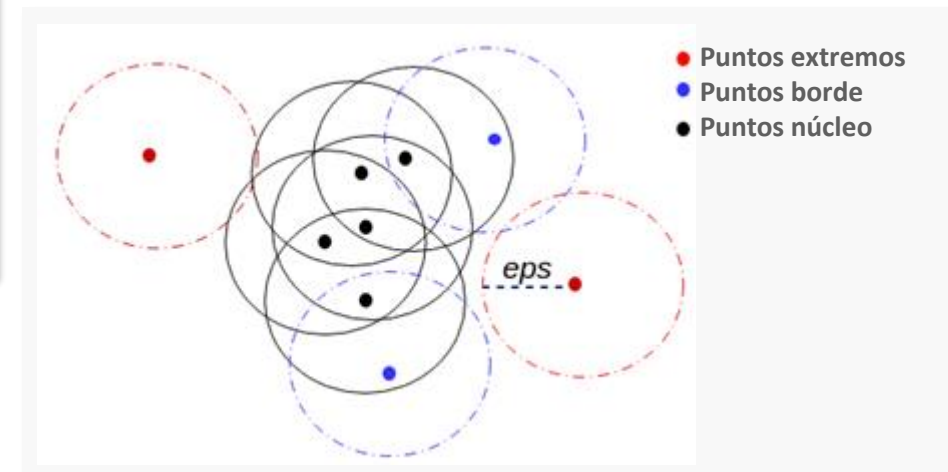
Crear grafos con los datos núcleos como nodos. Dos nodos estarán conectados si ellos están en la vecindad del otro.

Determinar los componentes conectados en el grafo (grupos)

Asignar cada dato borde al componente con el que esté mejor conectado.

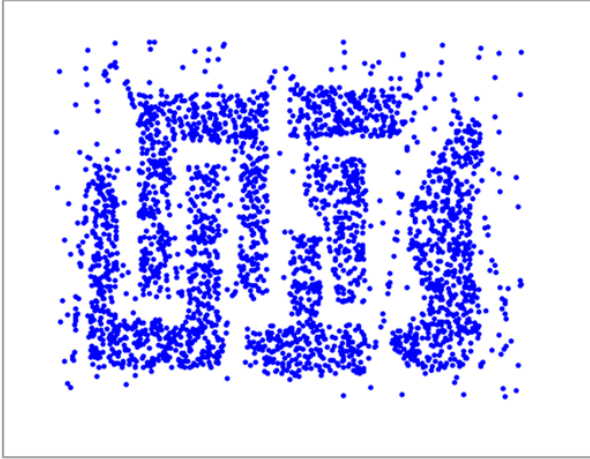
Fin-procedimiento

Salida = regiones conectadas (grupos), datos outliers

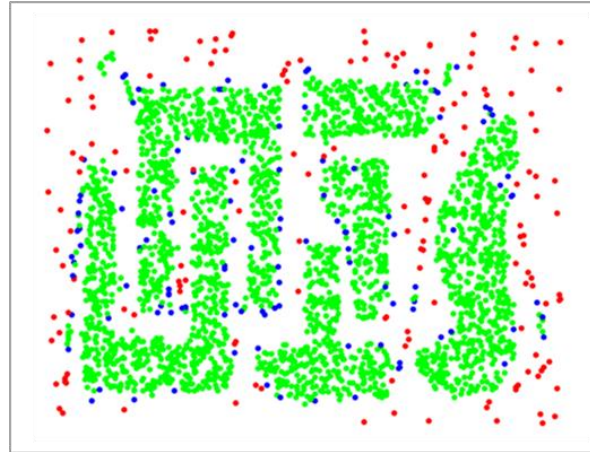


Ejemplo: $\epsilon = 10$, $minObj = 4$

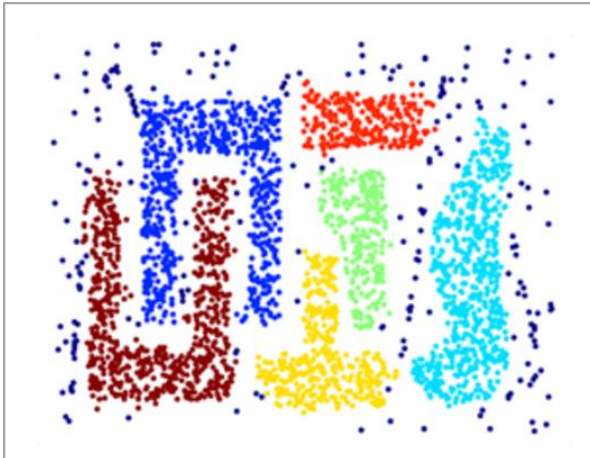
Conjunto de datos



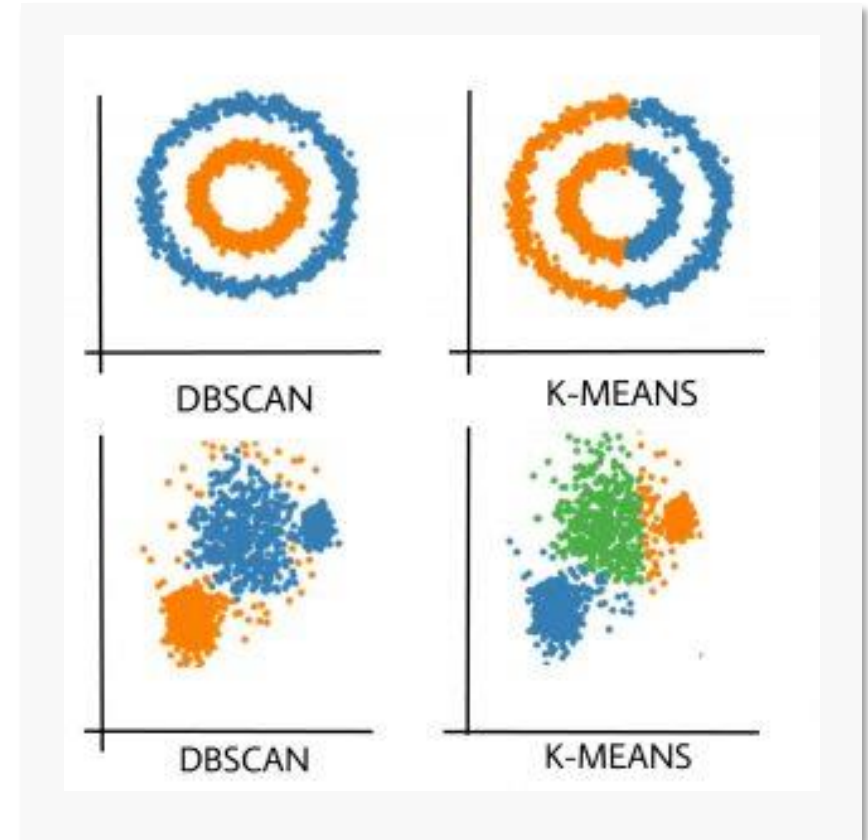
Tipos de datos: **núcleo**, **borde** y **outlier**



Grupos



Comparación:



<https://www.geeksforgeeks.org/dbscan-clustering-in-ml-density-based-clustering/>

En scikit-learn:

Parameters:

eps : *float, default=0.5*

The maximum distance between two samples for one to be considered as in the neighborhood of the other. This is not a maximum bound on the distances of points within a cluster. This is the most important DBSCAN parameter to choose appropriately for your data set and distance function.

min_samples : *int, default=5*

The number of samples (or total weight) in a neighborhood for a point to be considered as a core point. This includes the point itself. If `min_samples` is set to a higher value, DBSCAN will find denser clusters, whereas if it is set to a lower value, the found clusters will be more sparse.

metric : *str, or callable, default='euclidean'*

The metric to use when calculating distance between instances in a feature array. If metric is a string or callable, it must be one of the options allowed by `sklearn.metrics.pairwise_distances` for its metric parameter. If metric is "precomputed", X is assumed to be a distance matrix and must be square. X may be a [sparse graph](#), in which case only "nonzero" elements may be considered neighbors for DBSCAN.

⋮

Attributes:

core_sample_indices_ : *ndarray of shape (n_core_samples,)*

Indices of core samples.

components_ : *ndarray of shape (n_core_samples, n_features)*

Copy of each core sample found by training.

labels_ : *ndarray of shape (n_samples)*

Cluster labels for each point in the dataset given to fit(). Noisy samples are given the label -1.

n_features_in_ : *int*

Number of features seen during [fit](#).

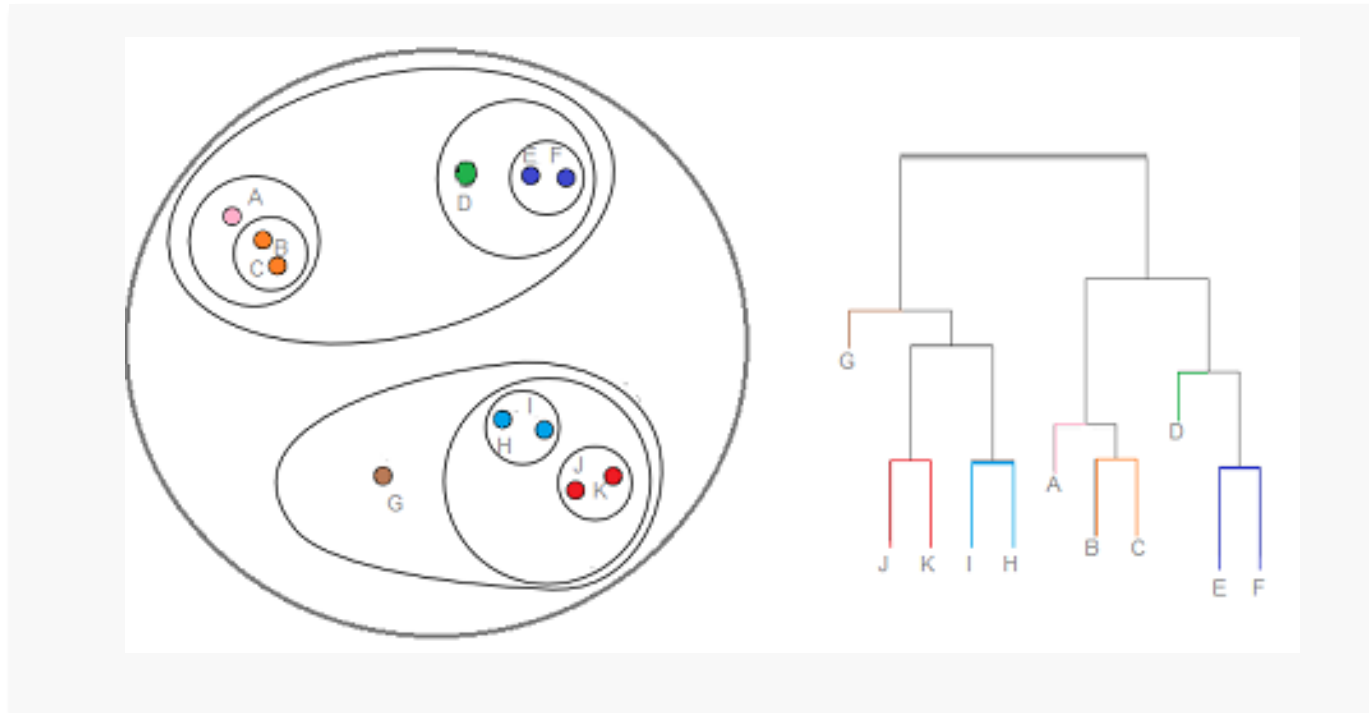
⋮

Ejemplo

```
>>> from sklearn.cluster import DBSCAN
>>> import numpy as np
>>> X = np.array([[1, 2], [2, 2], [2, 3],
...              [8, 7], [8, 8], [25, 80]])
>>> clustering = DBSCAN(eps=3, min_samples=2).fit(X)
>>> clustering.labels_
array([ 0,  0,  0,  1,  1, -1])
>>> clustering
DBSCAN(eps=3, min_samples=2)
```

❑ Agrupación jerárquica

- a) Criterio de división: grupos organizados en forma jerárquica.
 - b) Separación de los grupos: grupos solapados.
 - c) Medida de similitud: la similitud se determina con medidas de distancia.
-
- La jerarquía puede ser visualizada como un dendrograma.



- Dos tipos principales de métodos jerárquicos:

➡ Aglomerativos

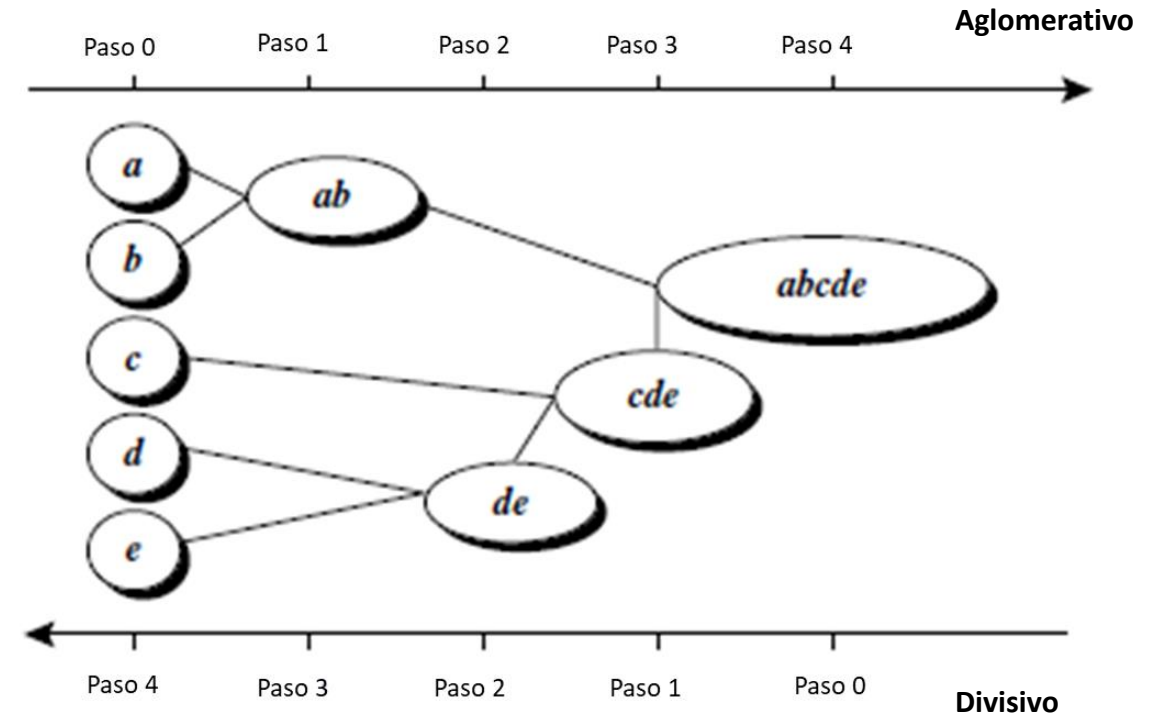
➡ Divisivos

Aglomerativos:

- Esquema *bottom-up*.
- Inicialmente cada dato es considerado un grupo.
- En cada iteración, se une el par de grupos más cercanos.
- Finaliza cuando sólo queda un grupo (raíz, con todos los datos) o cuando se haya alcanzado alguna otra condición de parada, como el número de grupos a determinar (especificado por el usuario).

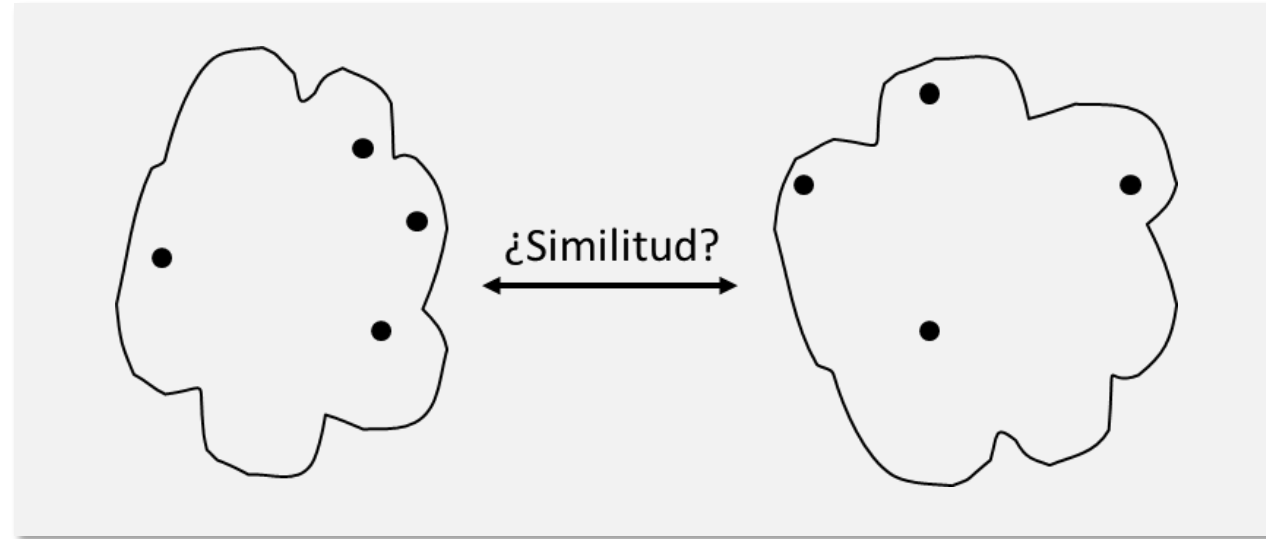
Divisivos:

- Esquema *top-down*.
- Inicialmente todo el conjunto de datos es un grupo (raíz).
- En cada iteración, se divide un grupo.
- Finaliza cuando cada grupo contiene un solo dato o se haya alcanzado alguna otra condición de parada, como el número de grupos a determinar (especificado por el usuario).



Importante: una vez que se han unido dos grupos o dividido un grupo no es posible rectificar en las siguientes iteraciones (no hay *swapping* de datos entre grupos)

¿Cómo determinar la similitud entre grupos?



☞ *distancia mínima* $\rightarrow dist_{min}(G_i, G_j) = \min_{x \in G_i, x' \in G_j} dist(x, x')$

☞ *distancia máxima* $\rightarrow dist_{max}(G_i, G_j) = \max_{x \in G_i, x' \in G_j} dist(x, x')$

☞ *distancia media* $\rightarrow dist_{media}(G_i, G_j) = dist(c_i, c_j)$

☞ *distancia promedio* $\rightarrow dist_{promedio}(G_i, G_j) = \frac{1}{n_i n_j} \sum_{x \in G_i, x' \in G_j} dist(x, x')$

Estas medidas de similitud son conocidas también como **medidas de enlace**

En scikit-learn:

Parameters:

n_clusters : *int or None, default=2*

The number of clusters to find. It must be `None` if `distance_threshold` is not `None`.

metric : *str or callable, default="euclidean"*

Metric used to compute the linkage. Can be "euclidean", "l1", "l2", "manhattan", "cosine", or "precomputed". If linkage is "ward", only "euclidean" is accepted. If "precomputed", a distance matrix is needed as input for the fit method. If connectivity is None, linkage is "single" and affinity is not "precomputed" any valid pairwise distance metric can be assigned.

Added in version 1.2.

memory : *str or object with the joblib.Memory interface, default=None*

Used to cache the output of the computation of the tree. By default, no caching is done. If a string is given, it is the path to the caching directory.

connectivity : *array-like, sparse matrix, or callable, default=None*

Connectivity matrix. Defines for each sample the neighboring samples following a given structure of the data. This can be a connectivity matrix itself or a callable that transforms the data into a connectivity matrix, such as derived from `kneighbors_graph`. Default is `None`, i.e, the hierarchical clustering algorithm is unstructured.

For an example of connectivity matrix using `kneighbors_graph`, see [Agglomerative clustering with and without structure](#).

⋮

linkage : *{'ward', 'complete', 'average', 'single'}, default='ward'*

Which linkage criterion to use. The linkage criterion determines which distance to use between sets of observation. The algorithm will merge the pairs of cluster that minimize this criterion.

- 'ward' minimizes the variance of the clusters being merged.
- 'average' uses the average of the distances of each observation of the two sets.
- 'complete' or 'maximum' linkage uses the maximum distances between all observations of the two sets.
- 'single' uses the minimum of the distances between all observations of the two sets.

Attributes:

n_clusters_ : *int*

The number of clusters found by the algorithm. If `distance_threshold=None`, it will be equal to the given `n_clusters`.

labels_ : *ndarray of shape (n_samples)*

Cluster labels for each point.

n_leaves_ : *int*

Number of leaves in the hierarchical tree.

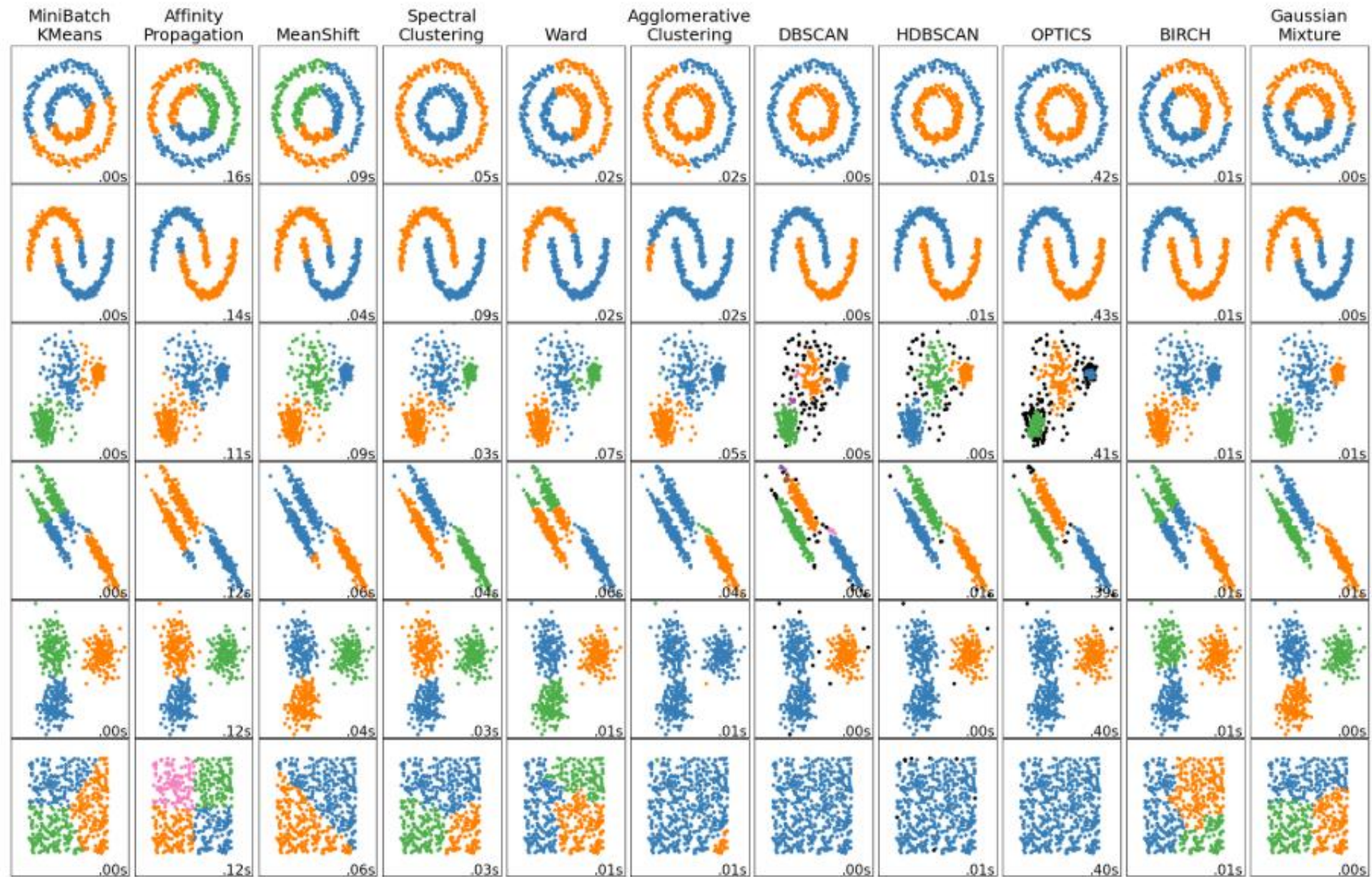
n_connected_components_ : *int*

The estimated number of connected components in the graph.

Ejemplo

```
>>> from sklearn.cluster import AgglomerativeClustering
>>> import numpy as np
>>> X = np.array([[1, 2], [1, 4], [1, 0],
...              [4, 2], [4, 4], [4, 0]])
>>> clustering = AgglomerativeClustering().fit(X)
>>> clustering
AgglomerativeClustering()
>>> clustering.labels_
array([1, 1, 1, 0, 0, 0])
```

❑ Otros algoritmos:





Próxima sesión

- Examen no. 1